unit - 1

client–Server Systems

Centralized server provides resources/services to multiple clients.
Server stores data & applications → clients (computers) access via network.
Examples: Bank database servers, email servers, company intranet.

Distributed Systems

Multiple independent computers connected and working together as one system.
Tasks are shared among computers for better performance and reliability.
Examples: Google Search servers, scientific simulations, blockchain networks.

Cloud Computing

Internet-based on-demand computing services (storage, applications, servers).
User doesn't need physical infrastructure.
Examples: Google Drive, AWS, Microsoft Azure.
Benefits: Scalability, Flexibility, Cost-effective.

Mobile Computing

Computing using portable devices like smartphones, tablets, laptops.
Features: Wireless communication (Wi-Fi, 4G/5G, Bluetooth), portability.
Examples: Online banking via mobile, GPS navigation, mobile apps.

4. Computer Languages

Computers only understand binary (0 and 1). To make programming easier, different generations of languages were developed.

1st Generation – Machine Language

Written in binary (0s and 1s).
Directly understood by CPU.
Example: 10110000 01100001 (represents a command).
Advantages: Fast execution.

Disadvantages: Difficult to learn, error-prone, not portable.

2nd Generation – Assembly Language

Uses mnemonics (symbols/short words) instead of binary.

Example: MOV A, B (move data from B to A).
Requires assembler to convert into machine code.
Easier than machine language but still hardware-specific.

3rd Generation – High-Level Languages (HLLs)

Close to human language, easier to understand.
Portable across different machines.
Examples: C, C++, Java, Python, FORTRAN, COBOL.
Requires compiler/interpreter to translate into machine code.

4th Generation Languages (4GLs)

Designed for non-programmers to write code quickly.
Mostly domain-specific (database, statistical, scripting).
Examples: SQL (for databases), MATLAB (for engineering), R (for statistics).
Require minimal coding effort, focus on problem-solving rather than syntax.

Summary Table – Computer Languages

Gen	Language Type	Example	Translator	Features
1st	Machine	0s & 1s	None	Fast, hard to code
2nd	Assembly	MOV A, B	Assembler	Mnemonics, hardware-dependent
3rd	High-Level	C, Java, Python	Compiler/Interpreter	Portable, easy, structured
4th	4GLs	SQL, MATLAB	Specialized Tools	Domain-specific, user-friendly

C language Introduction

The C Language is developed by Dennis Ritchie for creating system applications that directly interact with the hardware devices such as drivers, kernels, etc.

C programming is considered as the base for other programming languages, that is why it is known as mother language.

It can be defined by the following ways:

1.Mother language
2.System programming language
3.Procedure-oriented programming language
4.Structured programming language
5.Mid-level programming language

1) C as a mother language
C language is considered as the mother language of all the modern programming languages because most of the compilers, JVMs, Kernels, etc. are written in C language, and most of the programming languages follow C syntax, for example, C++, Java, C#, etc.

It provides the core concepts like the array, strings, functions, file handling, etc. that are being used in many languages like C++, Java, C#, etc.

2) C as a system programming language

A system programming language is used to create system software. C language is a system programming language because it can be used to do low-level programming (for example driver and kernel). It is generally used to create hardware devices, OS, drivers, kernels, etc. For example, Linux kernel is written in C.

It can't be used for internet programming like Java, .Net, PHP, etc.

3) C as a procedural language
A procedure is known as a function, method, routine, subroutine, etc. A procedural language specifies a series of steps for the program to solve the problem.

A procedural language breaks the program into functions, data structures, etc.

C is a procedural language. In C, variables and function prototypes must be declared before being used.

4) C as a structured programming language

A structured programming language is a subset of the procedural language. Structure means to break a program into parts or blocks so that it may be easy to understand.

In the C language, we break the program into parts using functions. It makes the program easier to understand and modify.

5) C as a mid-level programming language
C is considered as a middle-level language because it supports the feature of both low-level and high-level languages. C language program is converted into assembly code, it supports pointer arithmetic (low-level), but it is machine independent (a feature of high-level).

A Low-level language is specific to one machine, i.e., machine dependent. It is machine dependent, fast to run. But it is not easy to understand.

A High-Level language is not specific to one machine, i.e., machine independent. It is easy to understand.

Histroy

C programming language was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.

Dennis Ritchie is known as the founder of the c language.

It was developed to overcome the problems of previous languages such as B, BCPL, etc.

Initially, C language was developed to be used in UNIX operating system. It inherits many features of previous languages such as B and BCPL.

Let's see the programming languages that were developed before C language.

Language	Year	Developed By
1.Algol	1960	International Group
2.BCPL(Basic Combined Pro Language)	1966	Martin Richard
3.B	1969	Ken Thompson(developer of unix system)
4.Traditional C	1972	Dennis Ritchie(co - developer of unix system)

Features of C Language

C features
C is the widely used language. It provides many features that are given below.

Simple

Machine Independent or Portable
Mid-level programming language
structured programming language
Rich Library
Memory Management

Fast Speed
Pointers
Recursion

1) Simple
C is a simple language in the sense that it provides a structured approach (to break the problem into parts), the rich set of library functions, data types, etc.

2) Machine Independent or Portable
Unlike assembly language, c programs can be executed on different machines with some machine specific changes. Therefore, C is a machine independent language.

Machine Independent (Yes)

C is considered machine independent because a C program can be written on one machine and compiled on another (of the same architecture) without changing the source code.

Example: If you write a C program on Windows (Intel processor), the same source code can run on Linux (Intel processor) — you just need to recompile it.

Platform Independent (No)

C is not platform independent because it depends on the compiler and OS.
A program compiled on Windows generates a .exe file which will not run on Linux directly.
You must recompile the source code with a Linux compiler to generate a Linux-compatible binary.

3) Mid-level programming language
Although, C is intended to do low-level programming. It is used to develop system applications such as kernel, driver, etc. It also supports the features of a high-level language. That is why it is known as mid-level language.

1. Low-Level Language

Closest to machine hardware.
Difficult for humans to understand but very fast for machines.

Examples:
Machine language (binary – 0s & 1s)

Advantage: Fast execution, direct hardware control.
Disadvantage: Hard to learn, not portable.

2. Mid-Level Language

Acts as a bridge between low-level and high-level languages.
Provides access to both hardware-level operations (like pointers) and high-level features (like functions).

Example: C language

Can write system-level programs (like OS, device drivers).
Can also write application-level programs (like games, apps).

Advantage: Portable, efficient, allows hardware access.
Disadvantage: Requires good understanding of memory & hardware.

3. High-Level Language

Very close to human language.
Easy to learn and use.
Examples: Python, Java, C++, R, JavaScript.

4) Structured programming language
C is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify. Functions also provide code reusability.

5) Rich Library
C provides a lot of inbuilt functions that make the development fast.

6) Memory Management
It supports the feature of dynamic memory allocation. In C language, we can free the allocated memory at any time by calling the free() function.

7) Speed
The compilation and execution time of C language is fast since there are lesser inbuilt functions and hence the lesser overhead.

8) Pointer
C provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array, etc.

9) Recursion

In C, we can call the function within the function. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

```
#include <stdio.h>
int main(){
printf("Hello C Language");
return 0.0f;
}
```

General Overview of a Simple C Program's Structure:

The general architecture of a simple C program typically consists of several vital components. Below is an outline of the essential elements and their purposes:

Header Files:
The #include directives at the beginning of the program are used to include header files. Header files provide function prototypes and definitions that allow the C compiler to understand the functions used in the program.

Main Function:
Every C program starts with the main function. It is the program's entry point, and execution starts from here. The main function has a return type of int, indicating that it should return an integer value to the operating system upon completion.

Variable Declarations:
Before using any variables, you should declare them with their data types. This section is typically placed after the main function's curly opening brace.

Statements and Expressions:
This section contains the actual instructions and logic of the program. C programs are composed of statements that perform actions and expressions that compute values.

Comments:
Comments are used to provide human-readable explanations within the code. They are not executed and do not affect the program's functionality. In C, comments are denoted by // for single-line comments and /* */ for multi-line comments.

Functions:
C programs can include user-defined functions and blocks of code that perform specific tasks.

Functions help modularize the code and make it more organized and manageable.

Return Statement:
Use the return statement to terminate a function and return a value to the caller function. A return statement with a value of 0 typically indicates a successful execution in the main function, whereas a non-zero value indicates an error or unexpected termination.

Standard Input/Output:
C has library functions for reading user input (scanf) and printing output to the console (printf). These functions are found in C programs and are part of the standard I/O library (stdio.h header file). It is essential to include these fundamental features correctly while writing a simple C program to ensure optimal functionality and readability.

#include <stdio.h>:

In this line, the program includes the standard input/output library (stdio.h) due to the preprocessor directive. For input and output tasks, the stdio.h library contains methods like printf and scanf.

int main() { ... }:

It is the main function which is the entry point of the C program. The program starts executing from the beginning of the main function.

printf("Hello World!\n");

Use the printf() function to print formatted output to the console. In this example, the string "Hello, C Language" is printed, followed by a newline character (n) which moves the pointer to the following line after the message is displayed.

return 0;

When the return statement is 0, the program has been completed. When determining the state of a program, the operating system frequently uses the value returned by the main function. A return value of 0 often indicates that the execution was successful.

After compilation and execution, this C program will quit with a status code 0 and output "Hello, C Language" to the terminal.

Comments in C

In C programming, comments are used to make code easier to read and understand. Comments are used to provide information about lines of code. Programmers help document their work by providing descriptions of variables, methods, classes, or particular code segments. The compiler ignores comments, so they have no impact on how the program executes. Comments can be defined via double forward slashes (//).

Types of C Comments

There are mainly 2 types of comments in the C language.

1.Single Line Comments
2.Multi-Line Comments

Here, we will discuss these types of comments one by one.

Single Line Comments

In C, comments have a double forward slash (//) and are written as a single line. It has no effect on how the program executes and the compiler ignores everything placed on the line that follows //. Single-line comments are commonly used during testing or debugging to highlight certain code parts, provide brief explanations, or temporarily disable particular lines of code.

Multi-Line Comments

In C, multiple lines of code can be described or detailed explanations can be given using multi-line comments in C++. Multi-Line comments are represented by slash asterisk * ... *\. It can occupy many lines of code, but it can't be nested.

C Format Specifier

In C programming, the format specifier is a string that is used in the formatted input and output functions. The format string always starts with a '%' character and is utilized in several functions, such as printf(), scanf(), and many others. C programming language provide several format specifiers along with several different types, including %u for unsigned integer, %i for integer, %c for character, %f for float, and many others.

Types of Format Specifiers in C

There are several types of format specifiers that are commonly used in C. Some of them are as follows:

Format specifier
Description
1.%d or %i
It is used to print the signed integer value where signed integer means that the variable can hold both positive and negative values.

2.%u
It is used to print the unsigned integer value where the unsigned integer means that the variable can hold only positive value.

3.%o
It is used to print the octal unsigned integer in the octal integer value, where the octal number always represented with a leading 0 value.

4.%x
It is used to print the hexadecimal unsigned integer where the hexadecimal integer value always starts with a 0x value. In this, alphabetical characters are printed in small letters such as a, b, c, etc.

5.%X
It is used to print the hexadecimal unsigned integer, but %X prints the alphabetical characters in uppercase, such as A, B, C, etc.

6.%f
It is used for printing the decimal floating-point values. By default, it prints the 6 values after '.'.

7.%p
It is used to print the memory address in a hexadecimal form.

8.%c
It is used to print the signed character.

9.%s
It is used to print the strings.

10.%ld
It is used to print the long-signed integer value.

11.%%

It is used to print the % character itself.

12.%llu
It is mainly utilized to print the large unsigned integers.

What is a compilation?
The compilation is a process of converting the source code into object code. It is done with the help of the compiler. The compiler checks the source code for the syntactical or structural errors, and if the source code is error-free, then it generates the object code.

Compilation process in c
The c compilation process converts the source code taken as input into the object code or machine code. The compilation process can be divided into four steps, i.e., Pre-processing, Compiling, Assembling, and Linking.

The preprocessor takes the source code as an input, and it removes all the comments from the source code. The preprocessor takes the preprocessor directive and interprets it. For example, if <stdio.h>, the directive is available in the program, then the preprocessor interprets the directive and replace this directive with the content of the 'stdio.h' file.

The following are the phases through which our program passes before being transformed into an executable form:

Preprocessor
Compiler
Assembler
Linker

Compilation process in c

Preprocessor
The source code is the code which is written in a text editor and the source code file is given an extension ".c". This source code is first passed to the preprocessor, and then the preprocessor expands this code. After expanding the code, the expanded code is passed to the compiler.

Compiler
The code which is expanded by the preprocessor is passed to the compiler. The compiler converts this code into assembly code. Or we can say that the C compiler converts the pre-processed code into assembly code.

Assembler

The assembly code is converted into object code by using an assembler. The name of the object file generated by the assembler is the same as the source file. The extension of the object file in DOS is '.obj,' and in UNIX, the extension is 'o'. If the name of the source file is 'hello.c', then the name of the object file would be 'hello.obj'.

Linker

Mainly, all the programs written in C use library functions. These library functions are pre-compiled, and the object code of these library files is stored with '.lib' (or '.a') extension. The main working of the linker is to combine the object code of library files with the object code of our program. Sometimes the situation arises when our program refers to the functions defined in other files; then linker plays a very important role in this. It links the object code of these files to our program. Therefore, we conclude that the job of the linker is to link the object code of our program with the object code of the library files and other files. The output of the linker is the executable file. The name of the executable file is the same as the source file but differs only in their extensions. In DOS, the extension of the executable file is '.exe', and in UNIX, the executable file can be named as 'a.out'. For example, if we are using printf() function in a program, then the linker adds its associated code in an output file.

Let's understand through an example.

hello.c

```
#include <stdio.h>
int main()
{
   printf("Hello javaTpoint");
   return 0;
}
```

Compilation process in c

In the above flow diagram, the following steps are taken to execute a program:

Firstly, the input file, i.e., hello.c, is passed to the preprocessor, and the preprocessor converts the source code into expanded source code. The extension of the expanded source code would be hello.i.

The expanded source code is passed to the compiler, and the compiler converts this expanded source code into assembly code. The extension of the assembly code would be hello.s.

This assembly code is then sent to the assembler, which converts the assembly code into object code.

After the creation of an object code, the linker creates the executable file. The loader will then load the executable file for the execution.

```
#include<stdio.h>
int main(){
int number;
printf("enter a number:");
scanf("%d",&number);
printf("cube of number is:%d ",number*number*number);
return 0;
}
```

Tokens in C

In C, tokens are the most important element that is used to create a program in C. We can define the token as the smallest individual element in C. For example, we cannot make a sentence without using words; similarly, we cannot create a program in C without using tokens. Therefore, we can say that tokens in C are the building block or the basic component for creating a program in C language.

Classification of tokens in C

Tokens in C language can be divided into the following categories:

1.Keywords in C
2.Identifiers in C
3.Strings in C
4.Operators in C
5.Constant in C
6.Special Characters in C

Data Type

In C programming, a data type specifies the type of data that a variable can store, such as integers, floating-point numbers, characters, etc. It also determines the amount of memory allocated for the variable and the operations that can be performed on it. Like in most

programming languages, C uses data types to define and manage different sets of data. Generally, data types in C are classified into four main categories.

There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

a) Integer Types (Int)
The whole numbers, which are non-fractional and do not contain any decimal values, those numbers are stored under the int data type. We generally make use of the int data type for the variables, such as count, indices, and other variables. The int data type represents both positive and negative numbers as they are signed by default. An int takes up 4 bytes of memory on most devices, which allows it to store values between around -2 billion and +2 billion.

Signed Int: This data type has 4 bytes of memory storage used for storing both positive and negative integers. If we want to store such values, we use the signed int data type. If the system is a 32-bit system, it accepts values between the range of -2147483648 and 2,147,483,648.
// 9911338255
Example:

```
signed int x = -20;
```

Unsigned int: It stores only the positive integer values, i.e. between 0 and above, in a 4-byte memory storage. The range of the positive values is two times larger than the signed integer, i.e. from 0 to 4294967295, which is twice the signed integer range.

Example:

```
unsigned int x = 18;
```

Short int: If we want to store smaller integer values that mostly need two bytes of memory storage, we can use the short int. For the signed short int values, it has a limited range of -32768 to 32767. It saves memory in constrained environments.

Example:

```
short int a = 1000;
```

Long int: If we want to store larger numbers than a regular int, we can use the long int, which takes 4 or 8 bytes generally. When we compare on a 32-bit operating system, it can be compared with int data type.

Example:

```
long long int x = 1000000LL;
```

```
#include <stdio.h>

int main() {
    short int a = 45;
    unsigned short int b = 7545;
    int c = 2154455;
    unsigned int d = 441454581;
    long long int e = 24525366552LL;

    printf("short int a = %d \n", a);
    printf("unsigned short int b = %u \n", b);
    printf("int c = %d \n", c);
    printf("unsigned int d = %u \n", d);
    printf("long long int e = %lld\n", e);
    return 0;
}
```

b) Character Data Type (char)
In C, individual characters are represented by the char data type. It is typically used to hold ASCII or UTF-8 encoding scheme characters, such as letters, numbers, symbols, or commas. There are 256 characters that can be represented by a single char, which takes up one byte of memory. Characters such as 'A', 'b', '5', or '$' are enclosed in single quotes.

Syntax
It has the following syntax:

```
char ch = 'A';
```

C Character Type Example
Let us take an example to illustrate the character type in C.
Example

```
#include <stdio.h>
int main() {  //main function
  char word = 'T';
  printf("%c", word);
  return 0;
}
```

c) Float Data Type (Float)

The floating data type is used to represent floating-point integers. Floating numbers can be used to represent fractional units or numbers with decimal places. The float type is usually used for variables that require very good precision but may not be very precise. It can store values with an accuracy of about 6 decimal places and a range of about 3.4 x 1038 in 4 bytes of memory.

Syntax

It has the following syntax:

```
float variable_name;
```

C Float Data Type Example

Let us take an example to illustrate the float data type in C.

Example

```
#include <stdio.h>
int main() {   //main function
  float f_1 = 25.144501;
  printf("%f", f_1);
  return 0;
}
```

d) Double

Double data types is used to represent two floating integers. When additional precision is needed, such as in scientific calculations or financial applications, it provides greater accuracy compared to float.

Double type, which uses 8 bytes of memory and has an accuracy of about 15 decimal places, yields larger values. C treats floating point numbers as doubles by default if no explicit type is supplied.

Syntax

It has the following syntax:

```
double var_name;
```

C Double Data Type Example

Let us take an example to illustrate the double data type in C.

Example

```
#include <stdio.h>

int main() {  //main function
  double mynum = 17.578421;
  printf("%lf", mynum);
  return 0;
}
```

printf() and scanf() in C

The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio.h (header file).

printf() function

The printf() function is used for output. It prints the given statement to the console.

The syntax of printf() function is given below:

```
printf("format string",argument_list);
```

The format string can be %d (integer), %c (character), %s (string), %f (float) etc.

scanf() function

The scanf() function is used for input. It reads the input data from the console.

```
scanf("format string",argument_list);
```

Program to print cube of given number

Let's see a simple example of c language that gets input from the user and prints the cube of the given number.

Example

```
#include<stdio.h>
int main(){
int number;
printf("enter a number:");
scanf("%d",&number);
printf("cube of number is:%d ",number*number*number);
return 0;
```

```
}
```
The scanf("%d",&number) statement reads integer number from the console and stores the given value in number variable.

The printf("cube of number is:%d ",number*number*number) statement prints the cube of number on the console.

Program to print sum of 2 numbers
Let's see a simple example of input and output in C language that prints addition of 2 numbers.

Example
```
#include<stdio.h>
int main(){
int x=0,y=0,result=0;

printf("enter first number:");
scanf("%d",&x);
printf("enter second number:");
scanf("%d",&y);

result=x+y;
printf("sum of 2 numbers:%d ",result);

return 0;
}
```

C Boolean
In the C programming language, a boolean is a data type that contains two types of values, i.e., 0 and 1. Basically, the bool type value represents two types of behavior: either true or false. Here, '0' represents a false value, while '1' represents a true value.

In C Boolean, '0' is stored as 0, and another integer is stored as 1. We do not require the use of any header file to use the Boolean data type in C++ programming language, but in C, we have to use the header file, i.e., stdbool.h. If we do not use the header file, the program will not compile.

Boolean is supported by C99 and subsequent versions. By including<stdbool.h>, C added a boolean type with the standard C99. The <stdbool.h> is used in sample codes to use the bool type and true/false constants.

Syntax:
It has the following syntax:

```
bool variable_name;
```
In the above syntax,

bool: It represents the data type of the variable.
variable_name:It represents the name of the variable.

C Boolean Example
Let us take an example to illustrate the Boolean in C.

Example
```
#include <stdio.h>
#include<stdbool.h>
int main()    //main function
{
bool x=false; // variable initialization.
if(x==true) // conditional statements
{
printf("The value of x is true");
}
else
printf("The value of x is FALSE");
return 0;
}
```

C Variables

In C programming language, the term variable is used for holding different types of input data which is provided by the user. It is used to store information. Its value can be modified and reused several times. It is an important method that is used to represent memory location through symbols so that it can be easily identified.

In C, a variable is a designated memory region that stores a specified data type value. Each variable has a unique identifier, its name, and a data type describing the type of data it may hold.

Syntax

The syntax for defining a variable in C is as follows:

```
data_type variable_name;
```

In this syntax,

data_type:It represents the type of data (such as int, char, float, double, etc.) that the variable can hold.
variable_name:It is the identifier for the variable. It represents the name of the variable.

Simple Example of C Variable

Let us take an example to illustrate the variables in C.

Example

```
#include<stdio.h>
int main() {

int value1; // variable declared
value1=10; // variable initialized

char value2 = 'T'; // variable declared and initialized

float f1, f2, f3;
f1 = 10.9; // Initialized the floating variable f1
f2 = 3.5; // Initialized the floating variable f1
f3 = 6.1; // Initialized the floating variable f1

printf("The integer value stored in variable value1 is : %d \n \n ", value1);
printf("The character value stored in variable value2 is : %c \n \n ", value2);
printf("The float type values stored in variables f1, f2, f3 are : \n %f \n  %f \n  %f \n \n ", f1, f2, f3);
return 0;
}
```

In this example, we have created multiple variables that hold different types of data where Value1 holds the integer type of input value, Value2 holds the character type of input value, and F1, f2, and f3 hold the float type of input values. All these are the variable names we have used to store the user's input data in the memory without the need to remember the memory addresses for them.

Rules for Declaring a C Variable

In C, Variable names must follow a few rules to be valid. The following are the rules for naming variables in C:

1.Allowed Characters: Variable names include letters (uppercase and lowercase), digits, and underscores. They must start with a letter (uppercase or lowercase) or an underscore (_).

2.Case Sensitivity: C is a case-sensitive programming language. It means that uppercase and lowercase letters are considered distinct. For example, myVar, MyVar, and myvar are all considered different variable names.

3.Keywords: Variable names cannot be the same as C keywords (reserved words) because they have special meanings in the language. For example, we cannot use int, float, char, for, while, etc., as variable names.

4.Length Limitation: There is no standard limit for the length of variable names in C. It is better to keep them reasonably short and descriptive. Some compilers may impose a maximum length for variable names.

5.Spaces and Special Characters: Variable names cannot contain spaces or special characters (such as !, @, #, $, %, ^, &, *, (,), -, +, =, [,], {, }, |, \, /, <, >,., ?;, ', or "). Underscores are the only allowed special characters in variable names.

6.Reserved Identifiers: While not strictly a rule, it is advisable to avoid specific patterns or identifiers common in libraries or standard usage. For example, variables starting with __ (double underscores) are typically reserved for system or compiler-specific usage.

Valid and invalid variable names in C
There are several valid and invalid variable names. Some of them are as follows:

Valid Variable Names

```
int age;
float salary;
char _status;
double average_score;
int studentCount;
```

Invalid Variable Names

```
int 1stNumber; // Starts with a digit
float my-salary; // Contains a hyphen (-)
char int; // Same as a C keyword
int double; // Same as a C keyword
float my$var; // Contains an unsupported special character
```

Variable Declaration
In C programming, the process of telling the compiler about a variable's existence and its data type is known as variable declaration. It notifies the compiler that this particular variable with a

specific name and data type will be used in the program. Still, no memory for the variable will be allocated at that moment. It is usually seen at the start of a function or block before the variable is utilized.

Syntax

The general syntax for variable declaration is

data_type variable_name;

Example of Variable Declaration

Let us take an example to illustrate the variable declaration in C.

Example
```
#include <stdio.h>
int main() {
int age;
float salary;
char initial;

return 0;
}
```

Variable Definition

In C programming, the process of reserving memory space for a variable to keep its assigned value in the memory during program execution is known as a variable definition. It is based on the data type we have used for the variable, thus connecting the variable name with a particular memory address of sufficient size. A variable can be declared and defined in the same statement, although they can also be separated if necessary.

Variable Initialization

In C programming, the variable declaration is the act of informing the compiler about the existence and data type of a variable. It informs the compiler that a variable with a specific name and data type will be used in the program but that memory for the variable still needs to be allocated. Not explicitly, the initialized variables will contain garbage/random data that may result in unexpected program behavior.

Example of Variable Initialization

Let us take an example to illustrate the variable initialization in C.

Example

```
#include <stdio.h>
int main() {
   int age = 25;
   float salary = 2500.5;
   char initial = 'J';

   printf("Initial age: %d\n", age);
   printf("Initial salary: %.2f\n", salary);
   printf("Initial: %c\n", initial);

   age = 30;
   salary = 3000.0;

   printf("Updated age: %d\n", age);
   printf("Updated salary: %.2f\n", salary);

   return 0;
}
```

Type of C Variables

There are several types of variables in the C programming language. Some of them are as follows:

C Variables

Now, we will discuss every variable one by one.

Local Variable

In C programming, a local variable is a variable that is declared inside the method or block. A local variable must be declared at the start of the block. It means that we need to initialize a local variable before using it.

Example of Using Local Variable

Let us take an example to illustrate the local variable in C.

Example

```
#include <stdio.h>
int main( ) {
   int x=10;
```

```
    printf("The value assigned to the local variable is : %d ", x );
   return 0;
}
```
Output:

The value assigned to the local variable is : 10
Explanation:

In this example, we define a local integer variable x with a value of 10 inside the main() function. After that, it uses the printf() function to display the assigned value of x on the screen.

Global Variable

In C programming, a global variable is a variable that is declared outside the function or block. Any function can change the value of the global variable. It is available for all the functions. It must be declared at the start of the block.

Example of Using Global Variable

Let us take an example to illustrate the global variable in C.

Example
```
#include <stdio.h>
int value=20;
int main() {
int x=10;
printf("The value assigned to local variable is : %d\n", value );
printf("The value assigned to global variable is : %d", x );
}
```

In this example, we have taken the value as a global variable accessible throughout the program, while x is a local variable that is declared inside the main() function. After that, the printf function displays the value of both variables.

Static Variable

In C programming, a static variable is a variable that is declared with the 'static' keyword. A static variable retains its value between multiple function calls.

Example of Static Variable in C

Let us take an example to illustrate the use of a static variable in C.

Example

```
#include <stdio.h>
int main( ) {
int x=10;
static int y=10;
x=x+1;
y=y+1;
printf("The value assigned to local variable is : %d\n", x );
printf("The value assigned to static variable is : %d ", y );
}
```

Constants in C

In C programming language, constants are essential because they give us a mechanism to store unchanging values that hold true throughout the course of the program. These numbers may be used for several things, such as creating mathematical constants or giving variables set values.

In C programming, the constants are the read-only values that doesn't change as the program runs. Constants can be of several types, such as integers, floating-point numbers, characters, and strings. When a constant has a value, it cannot be changed, unlike variables. They may be utilized in various operations and computations and serve as the program's representation of fixed values.

Syntax

It has the following syntax:

```
const data_type var_name = value;
```

In this syntax,

const: It represents the const variable.
data_type: It represent the type of data.
var_name: It represents the name of the variable.

Example

```
#include <stdio.h>

int main() {  //Main Function
```

```
    const int MAX = 150;   // constant declaration
    printf("The value of MAX is: %d\n", MAX);

    return 0;
}
```

Literals in C

In C programming, literals are the constant values assigned to the constant variables. We can say that the literals represent the fixed values that cannot be modified. It also contains memory but does not have references as variables. Basic data types such as strings, characters, floats, and integers have underlying values referred to as literals. These values are assigned during compilation and remain unchanged throughout execution, hence the term "constants" or 'constant values'.

Keywords in C

Keywords in C are reserved words that have predefined meanings and are part of the C language syntax. These keywords cannot be used as variable names, function names, or any other identifiers within the program except for their intended purpose. They are used to define the structure flow and behavior of a C program.

The compiler recognizes a defined set of keywords in the C programming language. These keywords have specialized purposes and play critical roles in establishing the logic and behavior of a program. Here are some characteristics of C keywords:

Reserved: The C language reserves keywords are those keywords that cannot be used as identifiers in programs. Using a keyword as a variable name or other identifier will cause a compilation error.

Predefined Meaning: Each keyword has a specific meaning that is assigned by the C language. These meanings are built into the C language's grammar and syntax and the compiler interprets them accordingly.

Specific Use: Keywords are designed for specific purposes and contexts within the C language. They define control structures, data types, flow control, and other language constructs. Attempting to use a keyword outside of its intended purpose will result in a compilation error.

Standardized: C language keywords are standardized across different compilers and implementations. It ensures the consistency and portability of C programs across different platforms and environments.
A list of 32 keywords in the c language is given below:

auto	break	case	char	const					
continue	default	do	double	else					
enum	extern	float	for	goto					
if	int	long	register	return					
short	signed	sizeof	static	struct					
switch	typedef	union	unsigned	void					
volatile	while								

C Operators

In C programming language, operators are special symbols that are mainly used to perform operations on variables and values on operands. Operators are essential concepts of any programming language. There can be several operations, including arithmetic, value assignment, bitwise, and logical, relational computations. Operator allows us to handle and manipulate the data effectively. C has a large number of built-in operators that can be divided into different categories based on their utility.

Types of Operators
There are several types of operators in C. These operators are as follows:

Arithmetic Operators
Relational Operators
Logical Operators
Assignment Operators
Miscellaneous Operators

Now, we will discuss these operator one by one in C.

Arithmetic Operators

In C programming language, arithmetic operators carry out fundamental mathematical operations, such as addition (+), subtraction (-), multiplication (*), division (/), and modulus (%) on the operands. Several arithmetic operators in C are as follows:

Operator Name	Symbol	Description
Addition	+	The addition operator is used to add two operands together.
Subtraction	-	The subtraction operator is used to subtract the second operand from the first operand.
Multiplication	*	This operator is used to multiply the two operands.
Division	/	The first operand and the second operand are divided using the division operator.
Modulus	%	The modulus operator determines the remainder of the division between two operands.

Increment	++	It increments value by one x++ or ++x.
Decrement	--	It decrements value by one x-- or --x

Relational Operators

In C, relational operators are mainly used to compare two values or expressions and establish their relationship. These comparisons return a boolean value true or false (0 and 1). If the comparison is true, it returns 1 value. On the other hand, if the comparison is false, it returns 0 value. There are several operators in relational operators. Here is the following table that helps to understand the working of these operators in C:

Operator Name	Symbol	Description
Equal to	==	If two operands are equal, the equality operator verifies this.
Not Equal to	!=	The inequality operator determines whether two operands are equal or not.
Greater than	>	The greater than operator determines if the first operand exceeds the second operand.
Less than	<	The less-than operator determines if the first operand is less than the second operand.
Greater or equal	>=	The greater than or equal to the operator determines if the first operand is more than or equal to the second operand.
Less or equal to	<=	The less than or equal to the operator determines if the first operand must be less than or equal to the second operand.

Logical Operators

In C, logical operators are mainly used to perform logical operations on boolean values (true and false) and return either true (1) or false (0). These operators are essential for controlling the program flow based on conditions. If the comparison is true, it returns 1 value. If the comparison is false, it returns 0 value. Here is the following table that helps to understand the working of these operators in C.

Operator Name	Symbol	Description
Logical AND	&&	Called Logical AND operator. The logical AND operator returns true if both operands are true.
(Logical OR)	||	Called Logical OR Operator. The logical OR operator returns true if at least one of the operands is true.

(Logical NOT) !	Called Logical NOT Operator. The logical NOT operator negates the value of the operand.

Assignment Operators

In C, assignment operators are used to assign and update variable values. The basic assignment operator (=) stores the value of the right-hand operand in the left-hand variable, which makes it essential to variable initialization and updating. Here is the following table that helps to understand the working of these operators in C:

Operator Name	Symbol	Description
Assignment	=	It assigns the right operand to the left operand.
Addition Assignment	+=	It adds the right operand to the left and assigns the result there.
Subtraction Assignment	-=	It subtracts right from left and assigns the result to the left.
Multiplication Assignment	*=	Multiply left by right and allocate the result to the left.
Division Assignment	/=	It divides left by right and assigns the result to the left.
Modulus Assignment	%=	It computes the remainder and assigns it to the left.

Miscellaneous Operators

In C, miscellaneous operators refer to operators that do not fall under the arithmetic, relational, or logical category. They perform special operations, including memory access, type conversion, and conditional evaluation. Several miscellaneous operators in C as mentioned below:

Operator Name	Symbol	Description
sizeof Operator	sizeof	It returns the variable or data type's size (in bytes).
Conditional Operator (? :)	? :	Based on a criteria, one of two values is returned.
Comma Operator (,)	,	The comma operator analyses multiple expressions and returns the last one.
dot (.) and arrow (->) Operators. and -> objects and -> for object		Access structure members using. for direct pointers.
cast Operators	Type cast	It converts a data type to another explicitly.
addressof (&) Operators	&	Address-of operator. Returns the memory address of a variable.
Dereference (*) Operators	*	Pointer dereference operator. Accesses the value at the address specified by a pointer.

```c
#include <stdio.h>
struct Employee
{
    int age;
};
int main()
{
    int a = 10, b = 20, result;
    int *p = &a;
    printf("Size of int is: %lu\n", sizeof(int));
    result = (a > b) ? a : b;
    printf("Greater value using ? : is:  %d\n", result);
    result = (a = 5, b = 15, a + b);
    printf("Result using comma operator is: %d\n", result);

    struct Employee emp = {25};
    struct Employee *ptr = &emp;
    printf("Employee age using . is : %d\n", emp.age);
    printf("Employee age using -> is : %d\n", ptr->age);

    float x = 5.9;
    int y = (int)x;
    printf("After casting float to int: %d\n", y);

    printf("The address of a is: %p\n", (void*)&a);

    printf("Value pointed by p is: %d\n", *p);
    return 0;
}
```

1. Special Operators

These are operators with unique purposes:

1. sizeof: Returns the size (in bytes) of a data type or variable.
 sizeof(int) gives 4.

2. & (Address-of): Returns the memory address of a variable.

3. * (Pointer dereference): Accesses the value stored at an address.

4. ? : (Ternary operator): A shorthand for if-else.
result = (a > b) ? a : b;

5. , (Comma operator): Evaluates expressions from left to right.
6. -> and .: Access members of a structure.

2. Operator Precedence & Associativity

Precedence decides which operator executes first.
Associativity determines evaluation order when operators have the same precedence.

Example:
a + b * c → Multiplication has higher precedence than addition.
a = b = c → Right-to-left associativity.

3. Evaluation of Expressions

C evaluates expressions based on operator precedence, associativity, and parentheses.
Example:
int x = 5 + 2 * 3; → x = 11.

4. Type Conversions

C performs implicit (automatic) and explicit (manual) conversions:
Implicit: int to float automatically.
Explicit (Casting): (float)a / b.

Practical Example: C Program

```
#include <stdio.h>
int main() {
    int a = 5, b = 2;
    float result;
    printf("Size of int: %zu bytes\n", sizeof(int));
    int *ptr = &a;
    printf("Address of a: %p and value: %d\n", ptr, *ptr);
    int max = (a > b) ? a : b;
    printf("Max value using ?: operator: %d\n", max);
    int exp = a + b * 3;
    printf("Result of a + b * 3: %d\n", exp);
    result = (float)a / b;
    printf("Result of (float)a / b: %.2f\n", result);
```

```
    int x = (a = 10, b = 20, a + b);
    printf("Value using comma operator: %d\n", x);
    return 0;
}
```

unit-2

Control statement
if else
loop
switch
break
continue
goto

C if else Statement

In C programming, decision-making is a fundamental concept that allows a program to react differently in different circumstances. The if-else statement is used to implement the argument to decide. It enables the program to evaluate a situation and execute a block of the code, whether the problem is correct or wrong.

The if-else statement allows us to control the flow of execution. It helps the programmer to build dynamic and responsible applications that can handle real-world conditions user checking input, validating conditions, or reacting in various states in a program.

The if-else statement in C is used to perform the operations based on some specific condition. The operations specified in the if block are executed if and only if the given condition is true.

Types of if Statements in C++
There are several types of if statements in C language. Some of them are as follows:

If statement
If-else statement

If else-if ladder
Nested if

Each decision-making structure in C programming provides unique functionalities that boost programming flexibility because they perform distinct tasks.

1) if Statement
In C programming, the if statement is used to check some given condition and perform some operations depending upon the correctness of that condition. It is mostly used in scenarios where we need to perform different operations for various situations.

Syntax

It has the following syntax:

```
if (expression){
//code to be executed
}
```
In this syntax,

The if statement represents a conditional control structure that is used to execute a block of code only if the specified condition (expression) evaluates to true.

The expression inside the parentheses is evaluated:

If it returns true (non-zero), the code inside the {} block execute.

If it returns false (zero), the block is skipped.

if statement in c
C if Statement Example
Let us take a simple example to illustrate the if statement in C.

Example
```
#include<stdio.h>
int main(){      //main function
int number=0;
printf("Enter a number:");
scanf("%d",&number);
if(number%2==0){
printf("%d is even number",number);
```

```
}
return 0;
}
```
Output

```
Enter a number:4
4 is an even number
```
Another Example to find the largest number of the three

Let us take an example to find to largest number of three numbers using the if statement in C.

Example
```
#include <stdio.h>
int main()    //main function
{
   int a, b, c;
    printf("Enter three numbers: ");
   scanf("%d %d %d",&a,&b,&c);
   if(a>b && a>c)
   {
      printf("%d is largest",a);
   }
   if(b>a  && b > c)
   {
      printf("%d is largest",b);
   }
   if(c>a && c>b)
   {
      printf("%d is largest",c);
   }
   if(a == b && a == c)
   {
      printf("All are equal");
   }
}
```
Output

```
Enter three numbers: 12
23
34
34 is largest
```
Explanation:

In this example, we have taken three integers variables a, b, and c. After that, it asks the users to enter the numbers. Once the numbers are enetered, it checks the condition and provides the largest number among the three numbers.

Characteristics of If statement
Several characteristics of if statement in C language are as follows:

Objective: The if statement allows conditional execution of statement code on the basis of whether the expression given is correct.

Condition Evaluation: The condition inside the if statement should be an expression that returns a boolean result-the true (non-zero) or false (zero).

Flow Control: If the condition is true, the code block is executed. If the condition is false, the code block is skipped.

Efficiency: The if statement helps to avoid unnecessary computations or operations by executing code only when needed.

2) if-else Statement
In C programming, the if-else statement is used to perform two operations for a single condition. The if-else statement is an extension to the if statement, using which we can perform two different operations. If the condition is true, it executes the if block otherwise else block is executed.

3) If else-if ladder Statement
In C programming, the if-else-if ladder statement is an extension of the if-else statement. It is used in the scenario where multiple cases to be performed for different conditions. The if-else-if ladder statement executes one condition from multiple statements. Multiple cases can be handled through an if-else if conditional sequence in this structure. It is similar to the switch case statement, where the default is executed instead of the else block if none of the cases is matched.

4) Nested if Statement
In C, nested if statement structure includes another if statement within its body. Consecutive levels of conditional checks require the utilization of this statement.

C Loops

The looping can be defined as repeating the same process multiple times until a specific condition satisfies. There are three types of loops used in the C language. In this part of the tutorial, we are going to learn all the aspects of C loops.

Why use loops in C language?

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times. For example, if we need to print the first 10 natural numbers then, instead of using the printf statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantage of loops in C

1) It provides code reusability.
2) Using loops, we do not need to write the same code again and again.
3) Using loops, we can traverse over the elements of data structures (array or linked lists).

Types of C Loops

There are three types of loops in C language that is given below:

1.do while
2.while
3.for

For Loop in C

In C programming language, the for loop is used to iterate a task for a specific number of times. If the number of iterations is fixed, we can use the for loop instead of the while or do-while loop. It consists of three parts: initialization, condition, and increment or decrement.

Syntax

It has the following syntax:

```
for (initialization; condition; increment/decrement)
{
// Statements to repeat(loop body)
}
```

In this syntax,

Initialization: It is used to set the loop variable before it starts (e.g., int count = 1;). It just runs once.

Condition:The condition statement checks the given condition. If the condition is true, it executes the of for loop. If the condition is false, it terminates the for loop.

Update (Increment/Decrement): After execution of each loop, it adjusts the loop variable (e.g., count++) to help it reach the stopping point.

Loop Body: It represents the loop of the body. It continues to run as long as the condition holds true.

While Loop in C

The while loop is also known as a pre-tested loop. In general, a while loop allows a part of the code to be executed multiple times depending upon a given Boolean condition, as the condition passed in a while loop is of Boolean type. We can also view a while loop as a repeating if statement, which is mostly used in the case where the number of iterations is not known in advance.

Syntax of the While loop:
The syntax of the while loop in the C language is given below:

```
while( condition )
{
// code to be executed
// increment or decrement the variable
}
```

In this syntax, the condition is a Boolean expression that determines whether the loop continues to execute. We can specify this condition as needed, which is followed by the loop body containing the desired statements. Finally, the loop variable is updated (incremented or decremented) to control the loop's progression.

Flowchart of the while loop in C
Here is the flowchart of the while loop that shows how it works.

While Loop in C
Step 1: Start the program.

Step 2: Initialize a variable for the loop.

Step 3: Check for the condition provided in the while loop

Step 4: If the condition is true,

Execute the statements within the loop.
Update the loop variable.
Repeat the same.
Step 5: If the condition is false, exit the loop.

Step 6: End of the program.

do while loop in C

A loop is a programming control structure that allows you to execute a block of code indefinitely if a specific condition is met. Loops are used to execute repeating activities and boost programming performance. There are multiple loops in the C programming language, one of which is the "do-while" loop.

A "do-while" loop is a form of a loop in C that executes the code block first, followed by the condition. If the condition is true, the loop continues to run; else, it stops. However, whether the condition is originally true, it ensures that the code block is performed at least once.

do while loop syntax

The syntax of the C language do-while loop is given below:

```
do{
//code to be executed
}while(condition);
```

The components are divided into the following:

The do keyword marks the beginning of the Loop.

The code block within curly braces {} is the body of the loop, which contains the code you want to repeat.
The while keyword is followed by a condition enclosed in parentheses (). After the code block has been run, this condition is verified. If the condition is true, the loop continues else, the loop ends.

C break statement

The break is a keyword in C which is used to bring the program control out of the loop. The break statement is used inside loops or switch statement. The break statement breaks the loop one by one, i.e., in the case of nested loops, it breaks the inner loop first and then proceeds to

outer loops. The break statement in C can be used in the following two scenarios:

With switch case
With loop

C Continue Statement

In the C programming language, a continue statement is a jump statement that is used to skip the current iteration of any loop and continue with the next iteration. It is used in the for, while, and do-while looping techniques. It is commonly utilized to bypass the remaining statements in the current iteration without terminating the complete loop and moving to the next iteration.

goto Statement in C

The goto statement is known as jump statement in C. As the name suggests, goto is used to transfer the program control to a predefined label. The goto statment can be used to repeat some part of the code for a particular condition. It can also be used to break the multiple loops which can't be done by using a single break statement. However, using goto is avoided these days since it makes the program less readable and complecated.

Syntax:

```
label:
//some part of the code;
goto label;
```

goto Example

Example

```
#include <stdio.h>
int main()
{
  int num,i=1;
  printf("Enter the number whose table you want to print?");
  scanf("%d",&num);
  table:
  printf("%d x %d = %d\n",num,i,num*i);
  i++;
```

```
  if(i<=10)
  goto table;
}
```

C Arrays

In C programming, an array is defined as the collection of similar types of data items stored at contiguous memory locations. Arrays are the derived data type in C programming language that can store the primitive type of data, such as int, char, double, float, etc. It also has the capability to store the collection of derived data types, such as pointers, structure, etc. The array is the simplest data structure where each data element can be randomly accessed by using its index number.

C Arrays

In C programming, array is beneficial if we want to store similar elements. For example, if we want to store the marks of a student in 6 subjects, we don't need to define different variables for the marks in the different subjects. Instead of that, we can define an array that can store the marks in each subject at the contiguous memory locations.

By using the array, we can access the elements easily. Only a few lines of code are required to access the elements of the array.

Properties of an Array

The array contains the following properties:

Each element of an array is of the same data type and carries the same size, i.e., int = 4 bytes.

Elements of the array are stored at contiguous memory locations where the first element is stored at the smallest
memory location.

Elements of the array can be randomly accessed because we can calculate the address of each element of the array with the given base address and the size of the data element.

Declaration of C Array

We can declare an array in the C language in the following way.

```
data_type array_name[array_size];
```

Now, let us see the example to declare the array.

```
int marks[5];
```

Here, int is the data_type, marks are the array_name, and 5 is the array_size.

Initialization of C Array

The simplest way to initialize an array is by using the index of each element. We can initialize each element of the array by using the index. Consider the following example.

```
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
```

C Array Example

Let us take an example to illustrate an Array in C.

Example

```
#include<stdio.h>
int main(){      //main function
int i=0;
int marks[5];//declaration of array
marks[0]=80;//initialization of array
marks[1]=60;
marks[2]=70;
marks[3]=85;
marks[4]=75;
//traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}//end of for loop
return 0;
}
```

Output:

```
80
60
70
85
75
```

Explanation:

In this example, we declare an integer array of marks of size 5. Each element is initialized individually with values like 80, 60, 70, etc. A for loop is employed to loop through the array from index position 0 to 4 and print out the value using printf.

C Array Declaration with Initialization Example

Let's take an example to illustrate how to declare and initialize the array at the time of declaration in C.

Example

```
#include<stdio.h>
int main(){       //main function
int i=0;
int marks[5]={20,30,40,50,60};//declaration and initialization of array
 //traversal of array
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}
return 0;
}
```

Output:

```
20
30
40
50
60
```

Explanation:

In this example, we define and initialize a five-element integer array mark with values 20, 30, 40, 50, and 60. After that, we use a for loop to iterate over the array from index 0 through 4, which prints each of its elements using printf.

Accessing Array Elements

In C programming, array supports random access to its element, i.e., we can access any element of the array by giving the position of the element, referred to as the index. The values of the index begin from 0 and proceed up to array_size-1. We supply the index within square brackets [] followed by the name of the array.

C Arrays

Syntax

It has the following syntax:

```
array_name [index];
```

Where, the index value lies into this range - ($0 \leq index \leq size-1$).

C Example to Access Array Elements

Let us take an example to illustrate how to access array elements in C.

Example

```
#include <stdio.h>
int main() {  //main function
   int numbers[4] = {10, 20, 30, 40};
   printf("Accessing array elements:\n");
   printf("Element at index 0: %d\n", numbers[0]);
   printf("Element at index 1: %d\n", numbers[1]);
   printf("Element at index 2: %d\n", numbers[2]);
   printf("Element at index 3: %d\n", numbers[3]);
   return 0;
}
```

Output:

```
Accessing array elements:
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
```

Explanation:

In this example, we consist of the standard input-output library and a definition of the main function. It has an integer array numbers of size 4 with initial values 10, 20, 30, and 40. After that, it prints every element using printf by addressing them using their index from 0 to 3. The program terminates by returning 0, which shows the successful program execution.

Updating the Array Element

In C, we can change the value of array elements at index i quite the same way we access an element using the array square brackets [] and assignment operator (=).

Syntax

It has the following syntax:

```
array_name[i] = new_value;
```

C Example to Update the Array Elements

Let us take an example to illustrate how to update the array elements in C.

Example

```
#include <stdio.h>
int main() {   //main function

   int numbers[5] = {5, 10, 15, 20, 25};
```

```
    printf("Original value at index 2: %d\n", numbers[2]);
    numbers[2] = 99;
    printf("Updated value at index 2: %d\n", numbers[2]);
    return 0;

}
```

Explanation:

In this example, we declare an integer array of numbers of size 5. Next. it prints the initial value at index 2, which is 15. After that, it modifies the value at index 2 to 99 via assignment (numbers[2] = 99;) and prints the modified value.

Array Traversal

In C, array traversal is the operation that allows us to travel through each element of the array in a particular sequence. In C array traversal, we use loops to access each element of the array.

C Array Traversal Example

Let us take an example to illustrate the array traversal in C.

Example

```
#include <stdio.h>
int main() {   //main function
    int numbers[5] = {2, 4, 6, 8, 10};
    printf("Traversing the array:\n");
    for (int i = 0; i < 5; i++) {
        printf("Element at index %d: %d\n", i, numbers[i]);
    }
    return 0;
}
```

Explanation:

In this example, we initialize an integer array of numbers of size 5 containing the elements 2, 4, 6, 8, and 10. It employs a for loop from index 0 to 4 to iterate through the array. Within the loop, printf is employed to output the index and the respective array element. The program terminates by returning 0.

Size of Array

The size of the array is the number of elements stored in the array. We can get the size using the sizeof() operator.

The sizeof() operator gives the size in bytes. sizeof(arr) gives the total bytes of the array.

Each element in an array is of type int, which has a size of 4 bytes. Therefore, we can find the size of the array by dividing the total size in bytes by the byte size of a single element.

C Size of Array Example
Let us take an example to illustrate the size of array in C.

Example
```
#include <stdio.h>
int main() {   //main function
   int numbers[] = {10, 20, 30, 40, 50};
   int size = sizeof(numbers) / sizeof(numbers[0]);
   printf("Size of the array: %d\n", size);
   return 0;
}
```
Output:

Size of the array: 5
Explanation:

In this example, we declare an integer array of numbers of size 5. After that, it calculates the number of elements by dividing the overall memory size of the array (sizeof(numbers)) by the memory size of a single element (sizeof(numbers[0])). The value is assigned to the variable size and printed.

Two-Dimensional Array in C
A Two-Dimensional array, or 2D array in C, is an array possessing precisely two dimensions. It can be thought of as rows and columns arranged in a two-dimensional plane.

Syntax

```
return_type array_name[size1]
                [size2] .. [sizen];
```

2-Dimensional Example in C

Example
```
#include <stdio.h>
int main() {  //main function
   int matrix[2][3];
   int value = 1;
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 3; j++) {
         matrix[i][j] = value;
         value++;
```

```
        }
    }
    printf("2D Array:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
2D Array:
1 2 3
4 5 6
```

Explanation:

In this example, we declare a 2D integer array matrix of size 2 rows and 3 columns. An integer variable value is set to 1. In nested for loops, each of the array elements is assigned numbers from 1 to 6 in increasing order. After that, the outer loop works for rows, and the inner loop for columns. Once initialized, another pair of nested loops is used to print the array in column-row order.

Arrays and Pointers

In C, arrays and pointers are directly interlinked with each other in such a way that we can use pointers to execute all the operations possible by the array. The array name is a constant pointer to the first element of the array, and the array is reduced to the pointers and passed to the function.

Array and Pointers Example in C

Let us take an example to illustrate the array and pointers in C.

Example

```
#include <stdio.h>
int main() {   //main function
    int numbers[5] = {10, 20, 30, 40, 50};
    int *ptr = numbers;
    printf("Accessing array elements using pointer:\n");
    for (int i = 0; i < 5; i++) {
        printf("Element %d: %d\n", i, *(ptr + i));
```

```
    }
    return 0;
}
```
Output:

```
Accessing array elements using pointer:
Element 0: 10
Element 1: 20
Element 2: 30
Element 3: 40
Element 4: 50
```
Explanation:

In this example, we declare an integer array of numbers of size 5. After that, a pointer ptr is given the base address of the array, i.e., it is pointing to the first element. The for loop iterates from index 0 to 4 and, on each pass, accesses the element at that index using pointer arithmetic: *(ptr + i) loads the value at the i-th position within the array. The printf statement prints the index and its corresponding value.

C Array Example: Sorting an array

In the following program, we are using the bubble sort method to sort the array in descending order.

Example

```
#include<stdio.h>
void main ()
{
    int i, j,temp;
    int a[10] = { 10, 9, 7, 101, 23, 44, 12, 78, 34, 23};
    for(i = 0; i<10; i++)
    {
        for(j = i+1; j<10; j++)
        {
            if(a[j] > a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
    printf("Printing Sorted Element List ...\n");
```

```
    for(i = 0; i<10; i++)
    {
        printf("%d\n",a[i]);
    }
}
```

Output:

```
Printing Sorted Element List ...
101
78
44
34
23
23
12
10
9
7
```

Advantages of C Array

Several advantages of the array in C are as follows:

Code Optimization: It need less code to access the data.
Ease of traversing: By using the for loop, we can retrieve the elements of an array easily.
Ease of sorting: We need a few lines of code only to sort the elements of the array.
Random Access: We can access any element randomly using

Linear Search Algorithm

Searching is the process of finding some particular element in the list. If the element is present in the list, then the process is called successful, and the process returns the location of that element; otherwise, the search is called unsuccessful.

Two popular search methods are Linear Search and Binary Search. So, here we will discuss the popular searching technique, i.e., Linear Search Algorithm.

Linear search is also called as sequential search algorithm. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL.

It is widely used to search an element from the unordered list, i.e., the list in which items are not sorted. The worst-case time complexity of linear search is O(n).

Function

Function is a block of code that can perform a specific task. It consists of a set of programming statements that is enclosed by curly braces {}. A function can be called multiple times to provide reusability and modularity to the C program. In other words, we can say that the collection of functions creates a program. The function is also known as procedure or subroutine in other programming languages.

Function Declaration

In C programming, a function declaration gives details about a function's name, parameters, and return type, allowing the compiler to make use of the information before it is defined. It ensures that function calls are valid and type-safe even if the function is defined later in the program.

It has the following syntax:

return_type function_name(parameter_list);
In this syntax;

return_type: It represents the return type, such as int, char, double, string, void, etc.
function_name: It is used to represent the name of the function.
parameter_list: It is used to represent the name of the parameter.

Types of Functions in C

There are mainly two types of functions in C programming.

1) Library Functions

Library functions are the inbuilt functions in C that are grouped and placed in a common place called the library. Such functions are used to perform some specific operations. For example, printf is a library function used to print on the console.

2) User Defined Functions in C
In C programming, user-defined functions are the functions that are created by the C programmer so that the user can use them many times. It reduces the complexity of a big program and optimizes the code.

Types of User Defined Function in C
A function may or may not accept any argument. It may or may not return any value. Based on these facts, there are four different aspects of function calls.

function without arguments and without return value
ex-void functionName(){}

function without arguments and with return value
ex-float functionName(){ return 0.0f; }

function with arguments and without return value
ex-void functionName(int a,float f){}

function with arguments and with return value
ex-int functionName(int a){ return 0;}

C Strings

In C programming language, the string can be defined as the one-dimensional array of characters terminated by a null ('\0'). The character array or the string is used to manipulate text, such as word or sentences. Each character in the array occupies one byte of memory, and the last character must always be 0.

The termination character ('\0') is important in a string because it is the only way to identify where the string ends. When we define a string as char s[10], the character s[10] is implicitly initialized with the null in the memory. The '%s' is used as a format specifier for the string in C programming language.

Declaration
In C programming, the declaration of a string is as easy as declaring a one-dimensional array of type characters. The following is the syntax for declaring a string.

char string_name[size];

In the given syntax,

string_name: It represents can the name that is assigned to the string variable.
size: It is used to declare the length of the string, i.e., how many characters' strings will hold.

There are two ways to declare a string in c language.
1.By char array
2.By string literal

Using char array
Let's see the example of declaring string by char array in C language.

char ch[10]={'r', 'a', 'h', 'u', 'l'};
As we know, array index starts from 0, so it will be represented as in the figure given below.

C Strings
While declaring string, size is not mandatory. So we can write the above code as given below:

char ch[]={'R', 'a', 'h', 'u', 'h'};
Using string literal
We can also define the string by the string literal in C language. For example:

char ch[]="Rahul";
In such case, '\0' will be appended at the end of the string by the compiler.

Initialization
We can initialize a string either by using the list of characters or a string literal.

C String Functions
There are many important string functions defined in "string.h" library.

No.
Function
Description
1)
strlen(string_name)
returns the length of string name.

```
printf("Length of string is: %d",strlen(ch));
```

2)
strcpy(destination, source)
copies the contents of source string to destination string.

```
strcpy(newch2,ch);
   printf("Value of second string is: %s",newch2);
```

3)
strcat(first_string, second_string)
concats or joins first string with second string. The result of the string is stored in first string.

```
char ch[10]={'h', 'e', 'l', 'l', 'o', '\0'};
   char ch2[10]={'c', '\0'};
   strcat(ch,ch2);
   printf("Value of first string is: %s",ch);
```

4)
strcmp(first_string, second_string)
compares the first string with second string. If both strings are same, it returns 0.

5)

strrev(string)
returns reverse string.

```
#include<stdio.h>
#include <string.h>
int main(){
  char str[20];
  printf("Enter string: ");
  gets(str);//reads string from console
  printf("String is: %s",str);
  printf("\nReverse String is: %s",strrev(str));
 return 0;
}
```

6)
strlwr(string)
returns string characters in lowercase.

printf("\nLower String is: %s",strlwr(str));

7)
strupr(string)
returns string characters in uppercase.

Strcmp() function in C
The C standard library includes the strcmp() function for string comparison. In C programming, it is frequently used to compare two strings and is a component of the <string.h> header. A connection between the two strings is shown by the function's return value, which is an integer value.

The strcmp(first_string, second_string) function compares two strings and returns 0 if both strings are equal.

Pointer to the first string that will be compared, str1.

Pointer to the second string that will be compared, str2.

If str1 is lexicographically superior to str2, it returns an integer number larger than 0.

If str1 is lexicographically smaller to str2, it returns an integer value that is less than 0.

It returns 0 if str1 and str2 are equal.

Here, we are using gets()function, which reads string from the console.

Example
```
#include<stdio.h>
#include <string.h>
int main(){
  char str1[20],str2[20];
  printf("Enter 1st string: ");
  gets(str1);//reads string from console
  printf("Enter 2nd string: ");
  gets(str2);
  if(strcmp(str1,str2)==0)
      printf("Strings are equal");
  else
```

```
    printf("Strings are not equal");
 return 0;
}
```

unit-3

C Pointers

In C programming, a pointer is a variable that is used to store the address of another variable. This variable can be of type int, char, array, function, or any other pointer. It enables us to manipulate the data that is stored at the specific memory location without using its variables. The size of the pointer depends on the architecture. However, in a 32-bit architecture, the size of a pointer is 2 bytes. The symbol of an address is represented by a pointer (*) asterisk symbol.

Syntax
It has the following syntax:

```
data_type *var_name;
```

In this syntax,
data_type: It represents the data type that the pointer is pointing to.
var_name: It represents the pointer name.

Consider the following example to define a pointer which stores the address of an integer.

```
int n = 10;
int* p = &n; // Variable p of type pointer is pointing to the address of the variable n of type integer.
```

Declaring a pointer
The pointer in the C language can be declared using * (asterisk symbol). It is also known as an indirection pointer used to dereference a pointer.

```
int *a; //pointer to int
char *c; //pointer to char
```

Accessing Address to Pointers

In C++, accessing addresses using a pointer is an essential concept that enables efficient memory manipulation and dynamic data structures, such as the arrays linked lists, trees, and many others. The addressof operator (&) is mainly used to store the address of a variable. After that, this address can be assigned to the variable of the pointer to initialize it.

```
int num = 32;
int *ptr = #
```

Dereference a Pointer

Accessing the pointer itself will only provide us with the address that is contained in the pointer. For instance,

Example

```
#include <stdio.h>
int main() {  //main function
   int num = 42;
   int *ptr = #
   printf("Value of num using pointer: %d\n", *ptr);
   *ptr = 28;
   printf("New value of num after dereferencing: %d\n", num);
   return 0;
}
</stdio.h>
```

Output:

```
Value of num using pointer: 42
New value of num after dereferencing: 28
```

Explanation:

In this example, we declare an integer num with the value 42 and a pointer ptr that contains its address. After that, it prints the value of num via the pointer (*ptr), changes num's value to 28 by dereferencing, and prints the new value.

We need to first dereference the pointer to access the value contained at the memory address. It is achieved with the assistance of a dereferencing operator(*).

C Another Pointer Example

Let us take another example to illustrate the pointer in C.

Example

```
#include<stdio.h>
int main(){    //using main function
int number=50;
int *p;
p=&number;  //stores the address of number variable
printf("Address of p variable is %x \n", p);
printf("Value of p variable is %d \n", *p);
```

```
return 0;
}
</stdio.h>
```
Output:

```
Address of p variable is d86801c
Value of p variable is 50
```
Explanation:

In this program, a pointer p is mainly utilized to store the address of the variable number. After that, the *p dereferences the pointer to access the value stored at that address. Finally, it prints both the address and the value of the number using the pointer.

Pointer to array

In C programming, a pointer to an array is a type of pointer that is used to store the address of the first array element. It can be utilized to access and manipulate the array elements using the pointer arithmetic.

C Pointer to an Array Example

Let us take an example to illustrate a pointer to an array in C.

Example

```
#include <stdio.h>
int main() {   //main function
   int arr[5] = {10, 20, 30, 40, 50};
   int (*ptr)[5] = &arr;
   printf("First element: %d\n", (*ptr)[0]);
   printf("Second element: %d\n", (*ptr)[1]);
   return 0;
}
</stdio.h>
```
Output:

```
First element: 10
Second element: 20
```
Explanation:

In this example, we define an array of integers and a pointer ptr that refers to the entire array. After that, it prints and accesses elements through (*ptr)[index].

Pointer to a function

In C programming, a pointer to a function is used to store the address of a function. It allows us to call the function indirectly. It is beneficial for callbacks, dynamic function calls, and passing

the functions as arguments.

Syntax

It has the following syntax:

```
void show (int);
void(*p)(int) = &display; // Pointer p is pointing to the address of a function
```

C Pointer to a Function Example

Let us take an example to illustrate the pointer to a function in C.

Example

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
int main() {  //main function
    int (*funcPtr)(int, int);
    funcPtr = add;
    int result = funcPtr(7, 3);
    printf("Result: %d\n", result);
    return 0;
}
</stdio.h>
```

Output:

```
Result: 10
```

Explanation:

In this example, we declare a function add and call it through a function pointer functor. After that, the pointer stores the address of the add, and the function is called through the pointer.

Pointer to Structure

In C programming, a pointer to a structure enables us to access the structure members through dynamic memory. It is mainly used with dynamic data structures, such as trees, linked lists, and many more.

Syntax

It has the following syntax:

```
struct st {
    int i;
```

```
    float f;
}ref;
struct st *p = &ref;
```

C pointer to a Structure Example
Let us take an example to illustrate a pointer to a structure in C.

Example

```
#include <stdio.h>
struct Student {
    char name[20];
    int age;
};
int main() {    //main function
    struct Student s1 = {"Johnty", 20};
    struct Student *ptr = &s1;
    printf("Name: %s\n", ptr->name);
    printf("Age: %d\n", ptr->age);
    return 0;
}
</stdio.h>
```

Output:

```
Name: Johnty
Age: 20
```

Explanation:

In this example, we declare a structure Student and a variable s1. After that, a pointer ptr is given the address of s1, and members of the structure are accessed by ptr->name and ptr->age to display the values.

Size of Pointers

The length of a pointer in C varies with the machine's architecture (bit system), not what it points to.

On a 32-bit system, all pointers usually use 4 bytes.
On a 64-bit machine, all pointers normally consume 8 bytes.
The size does not change even with the type of data (int*, char*, float*, etc.). We can check it with the sizeof operator.

The reason for the same amount of space is that the pointers hold the memory addresses irrespective of what their type is. Since the space occupied by storing the addresses of various memory locations remains the same, the space occupied by one type of pointer will be the

same as the space occupied by other types of pointers.

Size of Pointers Example
Let us take an example to illustrate the size of pointers in C.

Example
```
#include <stdio.h>
int main() {  //main function
   int *intPtr;
   float *floatPtr;
   char *charPtr;
   double *doublePtr;
   printf("Size of int pointer: %zu bytes\n", sizeof(intPtr));
   printf("Size of float pointer: %zu bytes\n", sizeof(floatPtr));
   printf("Size of char pointer: %zu bytes\n", sizeof(charPtr));
   printf("Size of double pointer: %zu bytes\n", sizeof(doublePtr));
   return 0;
}
</stdio.h>
```
Output:

```
Size of int pointer: 8 bytes
Size of float pointer: 8 bytes
Size of char pointer: 8 bytes
Size of double pointer: 8 bytes
```
Explanation:

In this example, we declare pointers of various data types and print their sizes using sizeof. After that, it indicates that pointer size is system-dependent, regardless of the type it points to.

Addressof (&) Operator
The address of operator '&' returns the address of a variable. However, we need to use %u to display the address of a variable.

C Addressof (&) Operator Example
Let us take an example to illustrate the addressof (&) operator in C.

Example
```
#include<stdio.h>
int main(){    //main function
int number=50;
printf("value of number is %d, address of number is %u",number,&number);
return 0;
```

```
}
</stdio.h>
```
Output:

```
value of number is 50, address of number is 2617052868
```
Special Types of Pointers
There are 4 special pointer types used or referred to in various contexts:

1) NULL Pointer
In C programming, a pointer that is not assigned any value but NULL is known as the NULL pointer. If we don't have any address to be specified in the pointer at the time of declaration, we can assign a NULL value. It will provide a better approach.

Syntax

It has the following syntax:

```
int *p=NULL;
```
In the most libraries, the value of the pointer is 0 (zero).

2) Void Pointer
In C programming, void pointers are the type of pointers that don't contain any data type associated with them. They are also referred to as generic pointers because they can be used to point at any type and can be typecasted to any type.

```
#include <stdio.h>
int main() {   //main function
   int a = 21;
   float b = 3.74;
   void *ptr;
   ptr = &a;
   printf("Value of a using void pointer: %d\n", *(int *)ptr);
   ptr = &b;
   printf("Value of b using void pointer: %.2f\n", *(float *)ptr);
   return 0;
}
```

In this example, we use void pointer ptr to hold the address of an int followed by a float and then typecasts the pointer to the appropriate type to dereference and print the values of a and b.

C Pointer Arithmetic

The arithmetic operations that can be done on a pointer are known as the pointer arithmetic. They are not the same as the operations we normally use for mathematical calculations since limited operations can be done on pointers. Those operations include:

Increment/Decrement
Addition/Subtraction of Integer
Subtracting Two Pointers of the Same Type
Comparing/Assigning Two Pointers of the Same Type
Comparing/Assigning with NULL

Constant Pointers

In constant pointers, the memory address that is stored within the pointer is constant and can't be changed after it's been declared. It will always hold the same memory address.

Example

```
#include <stdio.h>
int main() {  //main function
   int a = 10, b = 20;
   int *const ptr = &a;
   printf("Value: %d\n", *ptr);
   *ptr = 15;
   printf("Updated Value: %d\n", *ptr);
   return 0;
}
```

</stdio.h>

In this example, we define a constant pointer ptr to an integer a. After that, it prints the value pointed to, changes the value using *ptr, and outputs the new value. The pointer cannot be redirected to another variable, such as b, so it will always point at the same location.

Advantage of Pointer

There are several advantage of pointer in C programming. Some of them are as follows:

1.In C programming, a pointer reduces the code and improves the performance. It is used to retrieve strings, trees, etc. and used with arrays, structures, and functions.

2.We can return multiple values from a function using the pointer.
3.It allows us to access any memory location in the computer's memory.

Pointer Program to swap two numbers without using the 3rd Variable

Let us take an example to swap two numbers without using the 3rd variable in C.

```
#include<stdio.h>
int main(){
int a=10,b=20;
int *p1=&a,*p2=&b;
printf("Before swap: *p1=%d *p2=%d",*p1,*p2);
*p1=*p1+*p2;
*p2=*p1-*p2;
*p1=*p1-*p2;
printf("\nAfter swap: *p1=%d *p2=%d",*p1,*p2);
return 0;
}
```

```
</stdio.h>
```

Output:

```
Before swap: *p1=10 *p2=20
After swap: *p1=20 *p2=10
```

Usage of pointer

There are many applications of pointers in C language. Some of them are as follows:

Dynamic Memory Allocation: In C language, we can dynamically allocate memory using malloc() and calloc() functions where the pointer is used.
Arrays, Functions, and Structures: Pointers are widely used in arrays, functions, and structures. It reduces the code and improves the performance.

Problems with Pointers

Pointers are prone to errors and have the following drawbacks:

Memory can be corrupted if a wrong value is passed to pointers.
Pointers are slightly harder to learn.
Pointers are primarily the cause of memory leaks in C.
Using pointers for access is relatively slower than variables in C.
Uninitialized pointers may result in a segmentation fault.

pointers to pointers

A pointer to a pointer is a variable that stores the address of another pointer.
It allows multiple levels of indirection, meaning you can access the data indirectly through more than one pointer.

```
 int a = 10;          // Variable
    int *ptr1 = &a;      // Pointer to int
    int **ptr2 = &ptr1;
```

Array of Pointers

An array of pointers is simply an array in which each element is a pointer — that is, it holds memory addresses instead of direct values.

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, c = 30;
    int *ptr[3];  // Array of 3 integer pointers
    ptr[0] = &a;
    ptr[1] = &b;
    ptr[2] = &c;
    for(int i = 0; i < 3; i++) {
        printf("Value of element %d = %d\n", i, *ptr[i]);
        printf("Address stored in ptr[%d] = %d\n\n", i, ptr[i]);
    }
    return 0;
}
```

Structures in C

In C programming, a structure is a user-defined data type that enables us to store the collection of different data types. Each element of a structure is called a member. Structures can simulate the use of classes and templates because it can store various information. The struct keyword is used to define the structure of a program.

Syntax

It has the following syntax:

```
struct structure_name
{
   data_type member1;
   data_type member2;
};
```

In this syntax,

structure_name: It represents the name of structure.
data_type: It represents the type of data.
member1, member2: It represents the member names.

Declaring Structure Variable

We can declare a variable for the structure so that we can access the member of the structure easily. There are two ways to declare structure variable:

1.By struct keyword within the main() function

```
#include<stdio.h>
struct StudentInfo{
        int id;
        char name[10];
        float marks;
};
int main(){
   struct StudentInfo s1 = {101, "Vikas",99.5f};
   printf("Student id %d \n",s1.id);
   printf("Student name %s \n",s1.name);
   printf("Student marks %f \n",s1.marks);
return 0;
}
```

2.By declaring a variable at the time of defining the structure.

Syntax 1: Define Structure Variables within the main() function

```
struct structure_name
```

```
 {
    // member declarations
};
struct structure_name var;
```

Syntax 2: Declaring Structure Variables through Definition

```
struct structure_name
{
// member declarations
} var1, var
```

Accessing Members of the Structure

There are two ways to access structure members:

Using . (member or dot operator)
Using -> (structure pointer operator)

Using Dot (.) Operator

In C programming, the dot (.) operator is mainly used to access or modify the data members of the structure. It is very useful when we utilized structure variables directly in C.

C Example to access the members of the structure using dot (.) operator
Let us take a simple example to illustrate how to access structure members using dot(.) operator in C language.

Example

```
#include<stdio.h>
#include <string.h>
struct employee
{   int id;
    char name[50];
}e1;  //declaring e1 variable for structure
int main( )    //main function
{
   //store first employee information
   e1.id=101;
   strcpy(e1.name, "John");//copying string into char array
```

```
   //printing first employee information
   printf( "employee 1 id : %d\n", e1.id);
   printf( "employee 1 name : %s\n", e1.name);
return 0;
}
</string.h></stdio.h>
```

Output:

```
employee 1 id : 101
employee 1 name : John
```

Using -> (Structure Pointer Operator)

In C programming, the -> operator is mainly used to access members of a structure using a pointer to that structure. If we have a pointer to a structure, we cannot utilize the dot (.) operator directly. Instead, we have to use the -> operator that dereferences the pointer and accesses the member in one step.

C Example to access the members of the structure using -> (Structure Pointer Operator)

Let us take a simple example to illustrate how to access structure members using the structure pointer operator in C language.

Example

```
#include <stdio.h>
struct Employee {
   int emp_id;
   char emp_name[45];
};

int main() {
   struct Employee e1 = {201, "Peter"};
   struct Employee *ptr = &e1;

   // Accessing members using structure pointer operator
   printf("Employee ID: %d\n", ptr->emp_id);
   printf("Employee Name: %s\n", ptr->emp_name);

   return 0;
}
</stdio.h>
```

Copying Structures

Copying is one of the most fundamental operations on structures in C. The assignment operator (=) can be used to assign one structure variable to another, just as it is for fundamental data types.

Example

```
#include <stdio.h>
struct emp
{
   int emp_Id;
   char depart_ment[3];
};
int main()
{
   struct emp e1 = {101, "DCA"};
   struct emp e2 = e1;
   printf("Employee ID is: %d\n", e2.emp_Id);
   printf("Department is: %s\n", e2.depart_ment);
   return 0;
}

</stdio.h>
```

Nested Structures in C

A nested structure means having one structure inside another.
It helps represent complex data — like a student having an address, or an employee having a department record.

Syntax:

```
struct Outer {
   int id;
   char name[30];
   struct Inner {
      char city[20];
      int pincode;
```

```
    } address;
};
```

A nested structure means having one structure inside another.
It helps represent complex data — like a student having an address, or an employee having a department record.

Nested Structure within a Structure

```
#include <stdio.h>
struct Address {
    char city[20];
    int pincode;
};
struct Student {
    int roll_no;
    char name[30];
    struct Address addr;  // Nested structure
};
int main() {
    struct Student s1 = {101, "Rahul Tiwari", {"Faridabad", 121002}};
    printf("Student Details:\n");
    printf("Roll No: %d\n", s1.roll_no);
    printf("Name: %s\n", s1.name);
    printf("City: %s\n", s1.addr.city);
    printf("Pincode: %d\n", s1.addr.pincode);
    return 0;
}
```

```
#include <stdio.h>
struct Date {
    int day, month, year;
};
struct Embployee {
    int id;
    char name[30];
    struct Date joining_date;
};
int main() {
```

```
    struct Employee e;
    printf("Enter Employee ID: ");
    scanf("%d", &e.id);
    printf("Enter Name: ");
    scanf("%s", e.name);
    printf("Enter Joining Date (dd mm yyyy): ");
    scanf("%d %d %d", &e.joining_date.day, &e.joining_date.month, &e.joining_date.year);
    printf("\nEmployee Details:\n");
    printf("ID: %d\n", e.id);
    printf("Name: %s\n", e.name);
    printf("Joining Date: %02d-%02d-%04d\n", e.joining_date.day, e.joining_date.month,
e.joining_date.year);
    return 0;
}
```

Arrays of Structures

An array of structures means storing multiple records (of the same structure type) together — similar to how you store multiple integers in an int array.

```
#include <stdio.h>
struct Student {
    int roll_no;
    char name[30];
    float marks;
};
int main() {
    struct Student s[3];
    for(int i = 0; i < 3; i++) {
        printf("Enter Roll No, Name, and Marks for Student %d:\n", i + 1);
        scanf("%d %s %f", &s[i].roll_no, s[i].name, &s[i].marks);
    }
    printf("\n--- Student Details ---\n");
    for(int i = 0; i < 3; i++) {
        printf("Roll No: %d | Name: %s | Marks: %.2f\n", s[i].roll_no, s[i].name, s[i].marks);
    }
    return 0;
}
```

Structures and Functions

You can pass structures to functions and also return structures from functions.

Ways to Pass Structures

Pass individual members
Pass entire structure by value
Pass structure by reference (using pointer)

```
#include <stdio.h>
struct Student {
    int roll_no;
    char name[30];
    float marks;
};
void display(struct Student s) {
    printf("Roll No: %d\n", s.roll_no);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);
}
int main() {
    struct Student s1 = {101, "Rahul", 89.5};
    display(s1);
    return 0;
}
```

Self-Referential Structures

*

These structures are necessary for dynamic memory allocation and are commonly used in advanced algorithms and data structures.

Self-Referential Structure Example in C

Let us take an example to illustrate self-referential structures in C.

Example

```
#include <stdio.h>
struct Node
{
    int d;
    struct Node* next;
};
int main()  {
    struct Node n1 = {10, NULL};
    struct Node n2 = {20, NULL};
```

```
    n1.next = &n2;
    printf("Node-1 Data is: %d\n", n1.d);
    printf("Node-2 Data is: %d\n", n1.next->d);
    return 0;
}
```

1,2,3,4,5,6,7,8,9,11,12,14,17,18,19,20,21,22,23,24,27,29,30,31,32,
33,34,36,39,40,41,42,43,44,45,46,47,48(31-10-2025-6thlect)

Union in C

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location. The union can also be defined as many members, but only one member can contain a value at a particular point in time.

Union is a user-defined data type, but unlike structures, they share the same memory location.

```
union abc
{
   int a;
   char b;
}
```

```
union abc
{
   int a;
char b;
}var;
int main()
{
  var.a = 66;
  printf("\n a = %d", var.a);
  printf("\n b = %d", var.b);
}
```

```
*/
```

Feature	Structure	Union
Memory	Each member has its own memory (sum of all)	All members share the same memory
Data Validity	All values valid together	Only last assigned value is valid
Size	Sum of all members	Size of largest member
Example	Student with roll, marks, name stored together	Student with only one active value at a time

Enum in C

The enum in C is also known as the enumerated type. It is a user-defined data type that consists of integer values, and it provides meaningful names to these values. The use of enum in C makes the program easy to understand and maintain. The enum is defined by using the enum keyword.

way to define the enum in C:

enum flag{integer_const1, integer_const2,.....integter_constN};

In the above declaration, we define the enum named as flag containing 'N' integer constants. The default value of integer_const1 is 0, integer_const2 is 1, and so on. We can also change the default value of the integer constants at the time of the declaration.

For example:

enum fruits{mango, apple, strawberry, papaya};
The default value of mango is 0, apple is 1, strawberry is 2, and papaya is 3. If we want to change these default values, then we can do as given below:

```
enum fruits{
mango=2,
apple=1,
strawberry=5,
papaya=7,
};
```

Enumerated type declaration

As we know that in C language, we need to declare the variable of a pre-defined type such as int, float, char, etc. Similarly, we can declare the variable of a user-defined data type, such as

enum. Let's see how we can declare the variable of an enum type.

Suppose we create the enum of type status as shown below:

enum status{false,true};

Now, we create the variable of status type:

enum status s; // creating a variable of the status type.

In the above statement, we have declared the 's' variable of type status.

To create a variable, the above two statements can be written as:

enum status{false,true} s;

In this case, the default value of false will be equal to 0, and the value of true will be equal to 1.

Let's create a simple program of enum.

```
#include <stdio.h>
 enum weekdays{Sunday=1, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday};
 int main()
{
 enum weekdays w; // variable declaration of weekdays type
 w=Monday; // assigning value of Monday to w.
 printf("The value of w is %d",w);
    return 0;
}
```

In the above code, we create an enum type named as weekdays, and it contains the name of all the seven days. We have assigned 1 value to the Sunday, and all other names will be given a value as the previous value plus one.

Output

Enum in C
Let's demonstrate another example to understand the enum more clearly.

```
#include <stdio.h>
 enum months{jan=1, feb, march, april, may, june, july, august, september, october, november,
december};
int main()
```

```
{
 for(int i=jan;i<=december;i++)
 {
 printf("%d, ",i);
 }
    return 0;
}
```

In the above code, we have created a type of enum named as months which consists of all the names of months. We have assigned a '1' value, and all the other months will be given a value as the previous one plus one. Inside the main() method, we have defined a for loop in which we initialize the 'i' variable by jan, and this loop will iterate till December.

Output

Enum in C

Why do we use enum?
The enum is used when we want our variable to have only a set of values. For example, we create a direction variable. As we know that four directions exist (North, South, East, West), so this direction variable will have four possible values. But the variable can hold only one value at a time. If we try to provide some different value to this variable, then it will throw the compilation error.

The enum is also used in a switch case statement in which we pass the enum variable in a switch parenthesis. It ensures that the value of the case block should be defined in an enum.

Let's see how we can use an enum in a switch case statement.

```
#include <stdio.h>
enum days{sunday=1, monday, tuesday, wednesday, thursday, friday, saturday};
int main()  {
   enum days d;
   d=monday;
   switch(d) {
      case sunday:
      printf("Today is sunday");
      break;
      case monday:
      printf("Today is monday");
      break;
      case tuesday:
      printf("Today is tuesday");
```

```
      break;
      case wednesday:
      printf("Today is wednesday");
      break;
      case thursday:
      printf("Today is thursday");
      break;
      case friday:
      printf("Today is friday");
      break;
      case saturday:
      printf("Today is saturday");
      break;
  }

   return 0;
}
```

Output

Enum in C

Some important points related to enum

The enum names available in an enum type can have the same value. Let's look at the example.

```
#include <stdio.h>

int main(void) {
  enum fruits{mango = 1, strawberry=0, apple=1};
    printf("The value of mango is %d", mango);
    printf("\nThe value of apple is %d", apple);
  return 0;
}
```

Enum in C

If we do not provide any value to the enum names, then the compiler will automatically assign the default values to the enum names starting from 0.

We can also provide the values to the enum name in any order, and the unassigned names will get the default value as the previous one plus one.

The values assigned to the enum names must be integral constant, i.e., it should not be of other

types such string, float, etc.

All the enum names must be unique in their scope, i.e., if we define two enum having same scope, then these two enums should have different enum names otherwise compiler will throw an error.

File Handling in C

In programming, we may require some specific input data to be generated several numbers of times. Sometimes, it is not enough to only display the data on the console. The data to be displayed may be very large, and only a limited amount of data can be displayed on the console, and since the memory is volatile, it is impossible to recover the programmatically generated data again and again. However, if we need to do so, we may store it onto the local file system which is volatile and can be accessed every time. Here, comes the need of file handling in C.

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program. The following operations can be performed on a file.

Creation of the new file
Opening an existing file
Reading from the file
Writing to the file
Deleting the file

Functions for file handling

There are many functions in the C library to open, read, write, search and close the file. A list of file functions are given below:

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position

8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

Opening File: fopen()

We must open a file before it can be read, write, or update. The fopen() function is used to open a file. The syntax of the fopen() is given below.

```
FILE *fopen( const char * filename, const char * mode );
```

The fopen() function accepts two parameters:

1.The file name (string). If the file is stored at some specific location, then we must mention the path at which the file is stored. For example, a file name can be like "c://some_folder/some_file.ext".
2.The mode in which the file is to be opened. It is a string.

We can use one of the following modes in the fopen() function.

Mode	Description
w	opens a text file in write mode
w+	opens a text file in read and write mode
wb	opens a binary file in write mode
wb+	opens a binary file in read and write mode
a	opens a text file in append mode
a+	opens a text file in read and write mode
ab	opens a binary file in append mode
ab+	opens a binary file in read and write mode
r	opens a text file in read mode
r+	opens a text file in read and write mode
rb	opens a binary file in read mode
rb+	opens a binary file in read and write mode

The fopen function works in the following way.

Firstly, It searches the file to be opened.
Then, it loads the file from the disk and place it into the buffer. The buffer is used to provide efficiency for the read operations.
It sets up a character pointer which points to the first character of the file.

Consider the following example which opens a file in write mode.

Example

```
#include<stdio.h>
void main( )
{
FILE *fp ;
char ch ;
fp = fopen("file_handle.c","r") ;
while ( 1 )
{
ch = fgetc ( fp ) ;
if ( ch == EOF )
break ;
printf("%c",ch) ;
}
fclose (fp ) ;
}
```

Writing File : fprintf() function

The fprintf() function is used to write set of characters into file. It sends formatted output to a stream.

Syntax:

```
int fprintf(FILE *stream, const char *format [, argument, ...])
```

Example

```
#include <stdio.h>
main(){
   FILE *fp;
//   fp = fopen("file.txt", "w");
   fp = fopen("file.txt", "a");
  fprintf(fp, "Hello file by fprintf...\n");
   fclose(fp);//closing file
}
```

Reading File : fscanf() function

The fscanf() function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Syntax:

```
int fscanf(FILE *stream, const char *format [, argument, ...])
```

Example

```
#include <stdio.h>
main(){
  FILE *fp;
  char buff[255];
  fp = fopen("Anant.txt", "r");
  while( fscanf(fp, "%s", buff) !=EOF )
  {
  printf("%s ", buff );
  }
  fclose(fp);
}
```

C File Example: Storing employee information

```
#include <stdio.h>
void main()
{
   FILE *fptr;
   int id;
   char name[30];
   float salary;
   fptr = fopen("emp.txt", "w+");
   printf("Enter the id\n");
   scanf("%d", &id);
   fprintf(fptr, "Id= %d\n", id);
   printf("Enter the name \n");
   scanf("%s", name);
   fprintf(fptr, "Name= %s\n", name);
   printf("Enter the salary\n");
   scanf("%f", &salary);
   fprintf(fptr, "Salary= %.2f\n", salary);
   fclose(fptr);
}
```

Writing File : fputc() function

The fputc() function is used to write a single character into file. It outputs a character to a stream.

Syntax:

int fputc(int c, FILE *stream)
Example:

Example
```
#include <stdio.h>
main(){
  FILE *fp;
  fp = fopen("file1.txt", "w");
  fputc('a',fp);
  fclose(fp);//
}
```
file1.txt

a

Reading File : fgetc() function

The fgetc() function returns a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

```
#include<stdio.h>
void main( )
{
FILE *fp ;
char ch ;
fp = fopen("file_handle.c","r") ;
while ( 1 )
{
ch = fgetc ( fp ) ;
if ( ch == EOF )
break ;
printf("%c",ch) ;
}
fclose (fp ) ;
}
```

fseek() Function in C

the fseek() function is used to move the file pointer to a given position within a file. It is also utilized for file-based input and output operations, and it is a component of the Standard I/O Library (stdio.h). File reading and writing usually require sequential operations.

In C, the fseek() function is also useful when we need to access a file at random, such as removing file headers, changing specific records, or jumping to particular sections within large files.

Syntax of the fseek() function

It has the following syntax:

```
int fseek(FILE *stream, long int offset, int whence);
```

Parameters

stream: It represents the pointer to the FILE object.
offset: It represents the number of bytes to move the file pointer.
whence: It represents the reference position from where the offset is applied. It can be:

- SEEK_SET: It is used to represent the beginning of the file.
- SEEK_CUR: It represents the current position of the file pointer.
- SEEK_END: It is used to represent the end of the file.

Return Value

If the operation is passed, it returns 0.
If the operation is failed, it returns a non-zero value (such as an incorrect offset or invalid pointer).

Example code using the fseek() function in C

Let us take an instance to demonstrate the fseek() function in C.

Example

```
#include <stdio.h>

int main() {   //main function
   FILE *file;
   char ch;
   file = fopen("example.txt", "r");
```

```
    if (file == NULL) {
        printf("Error opening the file.\n");
        return 1;
    }
    fseek(file, 4, SEEK_SET);
    ch = fgetc(file);
    printf("Character at 5th position: %c\n", ch);
    fclose(file);
    return 0;
}
```

Syntax:

```
int fgetc(FILE *stream)
```

Example:

Example

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char c;
clrscr();
fp=fopen("myfile.txt","r");

while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
fclose(fp);
getch();
}
```

ftell Function in C

In C programming, the ftell() function returns the current file position of the specified stream. We can use the ftell() function to get the total size of a file after moving the file pointer to the end of the file. We can use the SEEK_END constant to move the file pointer to the end of the file. This function handles writing, reading, and the movement of the file pointer provided by the Standard I/O Library (stdio.h) module.

The ftell() function makes it simple for programmers to read/write from and to a file, insert, and search a file because ftell() returns a long integer that tells how many bytes the file pointer is

moved from the beginning of the file.

Syntax

It has the following syntax:

long ftell(FILE *stream);
Parameters

stream: It represents a reference to the file stream's corresponding FILE object.
Return Value: If successful, it provides a long representing the current file position. In case of an error, it returns -1L.

Example code using ftell() in C
Let us take an example to illustrate the ftell() function in C.

Example

```
#include <stdio.h>

int main() {
   FILE *file;
   long position;
     file = fopen("example.txt", "w");
   if (file == NULL) {
      printf("Error opening file!\n");
      return 1;
   }
   fputs("Hello, world!", file);
   position = ftell(file);
   printf("Current file position after writing: %ld\n", position);
   fclose(file);

   file = fopen("example.txt", "r");
   if (file == NULL) {
      printf("Error opening file!\n");
      return 1;
   }

   char ch;
   for (int i = 0; i < 5; i++) {
      ch = fgetc(file);          // Read characters
      printf("%c", ch);
```

```
    }

    position = ftell(file);        // Get position after reading 5 characters
    printf("\nFile position after reading 5 characters: %ld\n", position);

    fclose(file);
    return 0;
}
```

C rewind() function

In the C programming language, the rewind() function sets the file pointer at the beginning of the stream. It is very useful if we use a stream several times. The rewind() function is part of the C standard I/O library and resets the file pointer to the beginning. The rewind() function is mainly utilized to clear the error and EOF indicators.

Syntax of the rewind() function

It has the following syntax:

```
void rewind(FILE *stream)
```

In this syntax,

FILE: It represents the file towards which the indicator will point.

unit -4

1. Finding Factorial

The factorial of a number n (written as n!) is the product of all positive integers less than or equal to n. For example, 5! = 5×4×3×2×1 = 120. Factorial is widely used in permutations, combinations, and mathematical computations. It can be calculated using iteration (loops) or recursion (a function calling itself). The base case for recursion is when n = 0 or 1, where factorial equals 1. Factorials grow rapidly, so large numbers may cause overflow in C.

```
#include <stdio.h>
int factorial(int n){
    if(n==0 || n==1)
        return 1;
    else
        return n * factorial(n-1);
}
int main(){
    int n;
    printf("Enter number: ");
```

```
    scanf("%d",&n);
    printf("Factorial of %d is %d", n, factorial(n));
    return 0;
}
factorial(5)
  factorial(4)
    factorial(3)
      factorial(2)
        factorial(1)
          return 1
        return 2*1 = 2    n * 1 = 2
      return 3*2 = 6    n * 2  = 6
    return 4*6 = 24    n * 6 = 24
  return 5*24 = 120     n *  24 =120
```

2. Fibonacci Series

The Fibonacci series is a sequence of numbers where each term is the sum of the two preceding terms. It starts with 0 and 1. Mathematically, F(n) = F(n-1) + F(n-2) with F(0)=0 and F(1)=1. This series appears in nature (like leaf arrangements) and computer algorithms (like dynamic programming). It can be generated using iterative loops or recursive functions. Iteration is preferred as it avoids redundant recursive calls.

```
#include <stdio.h>
int main(){
    int n, t1=0, t2=1, next;
    printf("Enter number of terms: ");
    scanf("%d",&n);
    printf("Fibonacci Series: ");
    for(int i=1;i<=n;i++){
        printf("%d ",t1);
        next = t1 + t2;
        t1 = t2;
        t2 = next;
    }
    return 0;
}
```

3. Linear Search Algorithm

The linear search is a sequential searching algorithm. In linear search, we try to find a particular element (key) within the input array by traversing the array from the beginning and comparing each element with the key.

If the element is found in the array, the search is successful; if not, then the search is unsuccessful. The search is performed in O(n) time complexity in the worst-case scenario (when the element is at the very end or not present at all), and O(1) in the best case (when the element is the first one).

Linear Search Algorithm (Pseudocode)

```
LinearSearch(arr, size, keyVal)
  Set i = 0 (Start at the first element, index 0).
  Repeat while i < size
    If arr[i] == keyVal
      Return i (Element found at this index)
    [end of if]
    Increment i by 1
  [end of loop]
  Return -1 (Element not found)
```

Steps of Linear Searching

Firstly, we have to traverse the array elements using a for loop.
In each iteration of the for loop, compare the search element with the current array element, and:
If the element matches, then return the address of the corresponding array elements.
If the element does not match, then move to the next element.
If there is no match or the search element is not present in the array, return -1.

```
#include <stdio.h>
int Main(int arr[], int size, int keyVal) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == keyVal) {
            return i;
        }
    }
    return -1;
```

```
}
int main() {
    int arr[] = {76, 23, 45, 27, 67, 89, 12, 34};
    int size = sizeof(arr) / sizeof(arr[0]);
    int keyVal = 67;
    int res = Main(arr, size, keyVal);
    if (res != -1) {
        printf("Value %d found at index %d\n", keyVal, res);
    } else {
        printf("Value %d not found\n", keyVal);
    }
    return 0;
}
```

4.Binary Search Java

Binary search is an efficient algorithm used to search target element in a sorted array or a list. It follows the divide and conquer approach. It is faster than linear search. Note that an array or list must be sorted before performing the binary search. If we have an unsorted array or list, we can sort the array using the Arrays.sort(arr) method.

Working of Binary Search

1.Sorted Data Requirement: First, check if the given array or list is sorted in ascending order, or not. If not sorted, it must be sorted first.
2.Find the Middle Element: Find the middle element of the array or list to compare the target value.
3.Compare and Reduce:

If the target value matches the middle element, its position is found, and the search terminates.

If the target value is less than the middle element, the search continues in the left subarray.

If the target value is greater than the middle element, the search continues in the right subarray.

4.Repeat: Steps 2 and 3 are repeated on the new, reduced search space until the target value is found or the search space becomes empty (indicating the target is not present).

Binary Search Algorithm

```
BinarySearch(array, target):
    start = 0
    end = length(array) - 1
    while start <= end:
```

```
        mid = (start + end) // 2
        if array[mid] == target:
            return mid  // Target found at index mid
        else if array[mid] < target:
            start = mid + 1  // Search right half
        else:
            end = mid - 1    // Search left half
    return -1  // Target not found
```

```
#include <stdio.h>
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
int main() {
    int arr[20], n, key, result;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);
    result = binarySearch(arr, n, key);

    if (result == -1)
        printf("Element not found in array.");
    else
        printf("Element found at position %d.", result + 1);
    return 0;
```

}

Sorting Algorithms

Sorting is the process of arranging the elements of an array so that they can be placed either in ascending or descending order. For example, consider an array A = {A1, A2, A3, A4, ?? An }, the array is called to be in ascending order if element of A are arranged like A1 > A2 > A3 > A4 > A5 > ? > An .

Consider an array;

int A[10] = { 5, 4, 10, 2, 30, 45, 34, 14, 18, 9)

The Array sorted in ascending order will be given as;

A[] = { 2, 4, 5, 9, 10, 14, 18, 30, 34, 45 }

Bubble Sort Algorithm C

The Bubble Sort algorithm is one of the simplest sorting algorithms in computer science. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process continues until the list is sorted. Its name comes from the way smaller elements "bubble" to the top of the list. It is not suitable for large data sets.

Bubble short is majorly used where:

1.Complexity does not matter
2.Simple and short code is preferred

Here's a step-by-step explanation:

1.Compare Adjacent Elements: Start from the first element. Compare it with the next one.
2.Swap if Necessary: If the current element is greater than the next, swap them.
3.Repeat for All Pairs: Do this for all adjacent elements in the list. After the first pass, the largest element will "bubble up" to its correct position.
4.Ignore the Last Sorted Elements: Repeat the process for the remaining unsorted elements, ignoring the last sorted ones.
5.Stop When No Swaps are Made: If in a complete pass no elements are swapped, the list is sorted.

```
Algorithm
bubbleSort(array)
n = length(array)
repeat
  swapped = false
  for i = 1 to n - 1
        if array[i - 1] > array[i], then
        swap(array[i - 1], array[i])
        swapped = true
        end if
   end for
   n = n - 1
 until not swapped
end bubbleSort
```

Steps of Bubble Sort
Step 1: Compare the first two elements of the input array.

Step 2: If the first element is greater than the second element, swap them; otherwise, swapping is not required.

Step 3: According to step 2, do the comparison and swapping of the next pair of elements of the input array (the second and the third element), i.e. if the second element is greater than the third element, do the swapping; otherwise, not.

Step 4: Continue this process till the end of the input array is reached. At this point, the largest element will be "bubble up" and get the end of the list.

Step 5: Repeat the steps from 1 to 4, excluding the last element, as it has already achieved its correct position.

```
#include<stdio.h>
#include<stdbool.h>
 void print(int a[], int n)
    {
    int i;
    for(i = 0; i < n; i++)     {
        printf("%d ",a[i]);
```

```
   }
   }
 void bubble(int a[], int n)    // 15,10,20,17,19
 {
   int i, j, temp;
   bool isSwapped = false;
   for(i = 0; i < n; i++)     {
      isSwapped = false;
      for(j = 0; j < n - i - 1; j++)      {
           if(a[j] > a[j + 1])        {
              isSwapped = true;
              temp = a[j];
              a[j] = a[j + 1];
              a[j + 1] = temp;
           }
       }
       if(!isSwapped)   {
          break;
       }
   }
 }
void main()    {
    int i, j,temp;
    int a[5] = { 15, 16, 11, 13, 14};
    int n = sizeof(a)/sizeof(a[0]);
    printf("Before sorting array elements are: \n");
    print(a, n);
    bubble(a, n);
    printf("\nAfter sorting array elements are: \n");
    print(a, n);
}
```

Insertion sort

Insertion Sort Algorithm

Insertion sorting works similarly to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed on the right side; otherwise, it will be placed on the left side. Similarly, all unsorted cards are taken and put in their exact place.

The same approach is applied in insertion sort. The idea behind the insertion sort is that first, it takes one element and iterates it through the sorted array. Although it is simple to use, it is not appropriate for large data sets, as the time complexity of insertion sort in the average case and worst case is O(n2), where n is the number of items. Insertion sort is less efficient than other sorting algorithms like heap sort, quick sort, and merge sort etc. Insertion sort is a stable sorting algorithm.

Steps
The simple steps of achieving the insertion sort are listed as follows -
Step 1: If the element is the first element, assume that it is already sorted. Return 1.
Step 2: Pick the next element and store it separately in a key.
Step 3: Now, compare the key with all elements in the sorted array.
Step 4: If the element in the sorted array is smaller than the current element, then move to the next element. Else, shift greater elements in the array towards the right.
Step 5: Insert the value.

Step 6: Repeat until the array is sorted.

```
#include <stdio.h>

void insert(int a[], int n)
{
   int i, j, temp;
   for (i = 1; i < n; i++) {
      temp = a[i];
      j = i - 1;
      while(j>=0 && temp <= a[j]) {
         a[j+1] = a[j];
         j = j-1;
      }
      a[j+1] = temp;
   }
}
void printArr(int a[], int n) /* function to print the array */
{
   int i;
   for (i = 0; i < n; i++)
      printf("%d ", a[i]);
}
int main()
{
   int a[] = {70, 15, 2, 51, 60};
```

```
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are: \n");
    printArr(a, n);
    insert(a, n);
    printf("\nAfter sorting array elements are: \n");
    printArr(a, n);
    return 0;
}
```

Selection Sort Algorithm

In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted into its appropriate position into the array. It is also the simplest algorithm. It is an in-place comparison sorting algorithm.

In this algorithm, the array is divided into two parts; the first is the sorted part, and the other is the unsorted part. Initially, the sorted part of the array is empty, and the unsorted part is the given array. The sorted part is placed at the left, while the unsorted part is placed at the right.

Selection sort is generally used when:

A small array is to be sorted
Swapping cost does not matter
It is compulsory to check all elements

Steps of Selection Sort Algorithm

Step 1: Find the smallest element and swap it with the first element of the input array. If the first element is the smallest, then swapping is not required.

Step 2: Find the smallest element from the remaining elements (the second smallest element) and swap it with the second element. Again, here also swapping is not required if the second smallest element is the second element of the array.

Step 3: Repeat the process for the remaining elements of the input array until all elements achieve the correct position.

```
#include <stdio.h>
void selection(int arr[], int n)
{
    int i, j, small;
    for (i = 0; i < n-1; i++)
```

```
   {
      small = i;
         for (j = i+1; j < n; j++)
         if (arr[j] < arr[small])
         small = j;

      if (small != i)
      {
      int temp = arr[small];
      arr[small] = arr[i];
      arr[i] = temp;
      }
   }
}
void printArr(int a[], int n) /* function to print the array */
{
   int i;
   for (i = 0; i < n; i++)
      printf("%d ", a[i]);
}
int main()
{
   int a[] = {65, 26, 13, 23, 12};
   int n = sizeof(a) / sizeof(a[0]);
   printf("Before sorting array elements are: \n");
   printArr(a, n);
   selection(a, n);
   printf("\nAfter sorting array elements are: \n");
   printArr(a, n);
   return 0;
}
```

Find the Square Root of a Number

The square root of a number is a value that, when multiplied by itself, gives the original number. In C, we can calculate the square root using the built-in sqrt() function from the <math.h> library. The function returns a double value. Before using it, the user must include <math.h> and compile using -lm (e.g., gcc program.c -lm). Alternatively, the square root can be found using iterative methods like Newton-Raphson or binary search, but the library function is the simplest and most accurate.

Program:

```c
#include <stdio.h>
#include <math.h>

int main() {
    double num, result;
    printf("Enter a number: ");
    scanf("%lf", &num);
    result = sqrt(num);
    printf("Square root of %.2lf = %.2lf\n", num, result);
    return 0;
}
```

2. Reverse the Order of an Array

Reversing an array means changing the order of its elements so that the first element becomes the last and vice versa. This can be achieved by swapping elements from both ends using a loop that runs up to the midpoint of the array. The time complexity is O(n/2), effectively O(n). It is useful in sorting algorithms, data manipulation, and stack operations. In C, we use loops and temporary variables to interchange the elements.

```c
#include <stdio.h>

int main() {
    int n, i, temp;
    printf("Enter size of array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    for(i = 0; i < n / 2; i++) {
        temp = arr[i];
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }

    printf("Reversed array: ");
    for(i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
```

```
}
```

3. Reverse a String

Reversing a string means rearranging its characters in the opposite order. In C, strings are arrays of characters ending with a null character '\0'. The process involves swapping the first and last characters, then moving inward until the middle of the string is reached. The <string.h> library provides strlen() to find string length. This operation is often used in palindrome checking, encryption, and text manipulation. It runs in O(n) time complexity.

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[100], temp;
    int i, len;
    printf("Enter a string: ");
    gets(str);
    len = strlen(str);
    for(i = 0; i < len / 2; i++) {
        temp = str[i];
        str[i] = str[len - i - 1];
        str[len - i - 1] = temp;
    }
    printf("Reversed string: %s\n", str);
    return 0;
}
```

Pre-processor directives in C are instructions that are processed before the actual compilation begins. They begin with a hash symbol # and control code inclusion, macro substitution, and conditional compilation.

Definition:
A macro in C is a preprocessor directive that defines a name (identifier) for a piece of code, constant value, or expression. It is handled by the C preprocessor, which replaces every occurrence of the macro with its definition before the actual compilation begins. Macros are created using the #define directive.

#define defines macros or constants (e.g., #define PI 3.14).

is used in macros to convert arguments into strings.

concatenates tokens in macros.

#ifdef and #ifndef check if a macro is defined or not.

#if, #else, and #endif allow conditional compilation based on expressions.

1. #define — Defines macros or constants

```
#include <stdio.h>
#define PI 3.14       // constant macro
#define SQUARE(x) (x * x)  // function-like macro

int main() {
    printf("Value of PI: %.2f\n", PI);
    printf("Square of 5: %d\n", SQUARE(5));
    return 0;
}
```

Output:

```
Value of PI: 3.14
Square of 5: 25
```

2. # — Converts arguments into strings (Stringizing Operator)

```
#include <stdio.h>
#define PRINT(x) printf(#x " = %d\n", x)

int main() {
    int a = 10;
    PRINT(a);   // converts variable name into string "a"
    return 0;
}
```

Output:

a = 10

3. ## — Concatenates tokens (Token-Pasting Operator)

```
#include <stdio.h>
#define JOIN(a, b) a##b

int main() {
    int num1 = 100;
    printf("Value: %d\n", JOIN(num, 1));  // becomes num1
    return 0;
}
```

Output:

Value: 100

4. #ifdef — Checks if a macro is defined

```
#include <stdio.h>
#define DEBUG  // comment this line to see change

int main() {
#ifdef DEBUG
    printf("Debug mode is ON\n");
#endif
    return 0;
}
```

Output (when DEBUG is defined):

Debug mode is ON

If you remove #define DEBUG, nothing will print.

5. #ifndef — Checks if a macro is NOT defined

```
#include <stdio.h>

#ifndef VERSION
```

```
#define VERSION 1.0
#endif

int main() {
    printf("Version: %.1f\n", VERSION);
    return 0;
}
```

Output:

```
Version: 1.0
```

6. #if, #else, #endif — Conditional compilation

```
#include <stdio.h>
#define VALUE 15

int main() {
#if VALUE > 10
    printf("VALUE is greater than 10\n");
#else
    printf("VALUE is 10 or less\n");
#endif
    return 0;
}
```

Output:

```
VALUE is greater than 10
```

Common C Standard Libraries include:

stdio.h — Input/output functions (printf, scanf)
stdlib.h — Memory and process control (malloc, exit)
assert.h — Debugging checks (assert)
math.h — Mathematical functions (sqrt, pow)
time.h — Time/date handling (time, ctime)
ctype.h — Character testing/conversion (toupper, isdigit)
string.h — String operations (strlen, strcpy)
stdarg.h — Variable argument handling in functions (va_list, va_start)

Math Functions in C:
Trigonometry, exponentiation, logarithms, rounding, and other mathematical operations are available in the Cs math library. These approaches are more suited for complex calculations than simple arithmetic programming. The header file <math.h> offers methods for performing mathematical operations such as sqrt(), pow(), ceil(), floor(), etc.

C Math Functions
There are various math methods.h header file. The commonly used functions of math.h header files are given below.

Here are some examples of common math function categories in C:

No.
Function
Description
1) ceil(number) rounds up the given number. It returns the integer value which is greater than or equal to given number.
2) floor(number) rounds down the given number. It returns the integer value which is less than or equal to given number.
3) sqrt(number) returns the square root of given number.
4) pow(base,exponent)returns the power of given number.
5) abs(number) returns the absolute value of given number.

C Math Example

```
#include<stdio.h>
#include <math.h>
int main(){
printf("\n%f",ceil(3.6));
printf("\n%f",ceil(3.3));
printf("\n%f",floor(3.6));
printf("\n%f",floor(3.2));
printf("\n%f",sqrt(16));
printf("\n%f",sqrt(7));
printf("\n%f",pow(2,4));
printf("\n%f",pow(3,3));
printf("\n%d",abs(-12));
 return 0;
}
```
Output:

4.000000
4.000000
3.000000
3.000000
4.000000
2.645751
16.000000
27.000000
12

Trigonometric Functions:
The trigonometric functions are also used in the math function. The sine, cosine, and tangent of an angle are calculated using the functions sin(), cos().

Other known mathematical functions besides trigonometric functions are:

exp(): returns a number's exponential value (ex).
log() calculates a number's natural logarithm (base e).
log10() computes a number's common logarithm (base 10).
pow() raises a number to a given power.

Rounding Functions:
Following are some rounding functions that used in math function:

ceil() rounds a number up to the nearest integer.
floor () reduces a number to the nearest integer.
round() returns the nearest integer to a floating-point number.

Other Mathematical Functions:
Following are some other mathematical functions that used in math function:

sqrt() calculates the square root of a number.
fabs() returns the absolute value of a number.
fmod() calculates the remainder of the division between two numbers.

<math.h> Header:
If you want to utilize these arithmetic functions in your C application, include the <math.h> header at the start of your source code. The header file provides the function prototypes and the math function definitions. You can use the following directive to incorporate the math. In your C application, include the following header: h>

c

```
#include <math.h>
```
After including this header, you can utilize the math functions in your program as necessary.

Example
```
#include <stdio.h>
#include <math.h>

int main() {
double x = 2.0;
double result = sqrt(x);
printf("The square root of %.2f is %.2f\n", x, result);
return 0;
}
```
Output:

The square root of 2.00 is 1.41
Explanation:

In this example, the <math.h> header lets you use the sqrt() function to calculate the square root of a value.

Note: Remember the potential precision difficulties in floating-point computations while working with math functions that generate floating-point numbers.

1. sqrt() - Square Root Function
sqrt() function determines an integer's square root.

Example
```
#include <stdio.h>
#include <math.h>
int main() {
double x = 25.0;
double result = sqrt(x);
printf("The square root of %.2f is %.2f\n", x, result);
return 0;
}
```
Output:
The square root of 25.00 is 5.00

2. pow() - Exponential Power Function
The pow() function raises a number to a given power.

Example

```
#include <stdio.h>
#include <math.h>
int main() {
double base = 2.0;
double exponent = 3.0;
double result = pow(base, exponent);
printf("%.2f raised to the power of %.2f is %.2f\n", base, exponent, result);
return 0;
}
```

Output:

2.00 raised to the power of 3.00 is 8.00

3. sin(), cos(), and tan() - Trigonometric Function

These formulas calculate an angle's sine,cosine, and tangent in radians.

Example

```
#include <stdio.h>
#include <math.h>
int main() {
double angle = 1.0; // in radians
double sin_val = sin(angle);
double cos_val = cos(angle);
double tan_val = tan(angle);
printf("For angle %.2f:\n", angle);
printf("Sine: %.2f\n", sin_val);
printf("Cosine: %.2f\n", cos_val);
printf("Tangent: %.2f\n", tan_val);
return 0;
}
```

Output:

For angle 1.00:
Sine: 0.84
Cosine: 0.54
Tangent: 1.56

4. exp() - Exponential Function

The exp() function determines the number's exponential value (ex).

Example

```
#include <stdio.h>
#include <math.h>
```

```
int main() {
double x = 2.0;
double result = exp(x);
printf("e raised to the power of %.2f is %.2f\n", x, result);
return 0;
}
```
Output:
e raised to the power of 2.00 is 7.39

5. log() and log10() - Logarithmic Functions
The log() function calculates the natural logarithm (base e) of a number, while log10() calculates the common logarithm (base 10).

Example
```
#include <stdio.h>
#include <math.h>
int main() {
double x = 100.0;
double natural_log = log(x);
double common_log = log10(x);
printf("Natural logarithm of %.2f is %.2f\n", x, natural_log);
printf("Common logarithm of %.2f is %.2f\n", x, common_log);
return 0;
}
```
Output:
Natural logarithm of 100.00 is 4.61
Common logarithm of 100.00 is 2.00

6. ceil() and floor() - Rounding Functions
The ceil() function rounds a number up to the nearest integer, while floor() rounds it down.

Example
```
#include <stdio.h>
#include <math.h>
int main() {
double x = 3.7;
double y = -2.3;
printf("Ceil of %.2f is %.2f\n", x, ceil(x));
printf("Floor of %.2f is %.2f\n", x, floor(x));
printf("Ceil of %.2f is %.2f\n", y, ceil(y));
printf("Floor of %.2f is %.2f\n", y, floor(y));
return 0;
```

```
}
```
Output:

```
Ceil of 3.70 is 4.00
Floor of 3.70 is 3.00
Ceil of -2.30 is -2.00
Floor of -2.30 is -3.00
```

7. fabs() - Absolute Value Function

The fabs() function returns the absolute value of a given number, which is its distance from zero.

Example

```
#include <stdio.h>
#include <math.h>
int main() {
double x = -5.0;
double result = fabs(x);
printf("The absolute value of %.2f is %.2f\n", x, result);
return 0;
}
```
Output:

```
The absolute value of -5.00 is 5.00
```

8. fmod() - Remainder Calculation Function

The remaining value of a two-number division is calculated using the fmod() function.

Example

```
#include <stdio.h>
#include <math.h>

int main() {
double dividend = 10.0;
double divisor = 3.0;
double remainder = fmod(dividend, divisor);
printf("The remainder of %.2f divided by %.2f is %.2f\n", dividend, divisor, remainder);
return 0;
}
```
Output:

```
The remainder of 10.00 divided by 3.00 is 1.00
```

9. sinh() and cosh() - Hyperbolic Sine and Cosine Functions

A given integer hyperbolic sine or cosine can be calculated using the sinh() and cosh() functions.

Example

```
#include <stdio.h>
#include <math.h>
int main() {
double x = 2.0;
double sinh_val = sinh(x);
double cosh_val = cosh(x);

printf("Hyperbolic sine of %.2f is %.2f\n", x, sinh_val);
printf("Hyperbolic cosine of %.2f is %.2f\n", x, cosh_val);

return 0;
}
```

Output:

```
Hyperbolic sine of 2.00 is 3.63
Hyperbolic cosine of 2.00 is 3.76
```

10. atan2() - Arc Tangent Function (2 Arguments)

Considering both of the arguments' signs, the atan2() function calculates the arctangent of the quotient of its two inputs.

Example

```
#include <stdio.h>
#include <math.h>
int main() {
double x = 1.0;
double y = 1.0;
double result = atan2(y, x);
printf("The arctangent of y=%.2f and x=%.2f is %.2f radians\n", y, x, result);
return 0;
}
```

Output:

```
The arctangent of y=1.00 and x=1.00 is 0.79 radians
```

11. round() - Rounding to Nearest Integer

When given a floating-point number, the round() function returns the closest integer.

Example

```
#include <stdio.h>
```

```
#include <math.h>

int main() {
double x = 3.6;
double y = 3.3;
printf("Rounded value of %.2f is %.2f\n", x, round(x));
printf("Rounded value of %.2f is %.2f\n", y, round(y));
return 0;
}
```

Output:

```
Rounded value of 3.60 is 4.00
Rounded value of 3.30 is 3.00
```